

Sistema CapacityAR

Documentación técnica consolidada del backend del MVP. Plataforma de aprendizaje adaptativo (5P + ITS con HITL) para insertar jóvenes de barrios vulnerables al sector IT en Argentina.

Repositorio	github.com/marcelocassiovieira/capacitaya
Producción	capacity-ar-ap.onrender.com
Stack	Python 3.12 · FastAPI · PostgreSQL (Neon) · Render
IA	Groq + Gemini con fallback automático a Mock
Fecha	2026-06-08
Documentos	13 secciones

Índice

1. Informe técnico (interactivo)	p. 3
2. Informe técnico (resumen)	p. 13
3. Scope entre equipos	p. 20
4. Backend onboarding	p. 25
5. Integración con IA	p. 27
6. Cobertura del modelo 5P	p. 30
7. Gap engine MVP	p. 32
8. Diseño del módulo de entrenamiento	p. 40
9. Diseño de recursos	p. 50
10. Matriz de accesos	p. 53

11. Guía para el frontend	p. 58
12. Curls de prueba de la API	p. 65
13. Backlog	p. 73

Capacity AR · Informe técnico

Plataforma de aprendizaje adaptativo (5P + ITS con HITL) para insertar jóvenes de barrios vulnerables al sector IT en Argentina. Estado del backend al cierre de iteración.

Python 3.12 · FastAPI

PostgreSQL · Neon

Render free tier

Groq + Gemini + Mock fallback

Producción: capacity-ar-ap.onrender.com

6 / 14

ÉPICAS CERRADAS

8

MÓDULOS DE DOMINIO

~27 h

INVERTIDAS

13–16 h

PARA CERRAR MODELO 5P

1 · Consigna 2 · Arquitectura 3 · Lo realizado 4 · Cobertura 5P 5 · Lo pendiente 6 · Brechas
7 · Decisiones 8 · Riesgos 9 · Próximos pasos 10 · Referencias

1 Consigna y alcance

La consigna pide implementar un **Sistema de Tutoría Inteligente (ITS)** articulado sobre el marco de las **5 P** con **Human-in-the-Loop**, orientado a inclusión sociolaboral en IT. El flujo se opera en cinco fases:

Fase 0

Diagnóstico de brecha

Fase 1

Pasión

Fase 2

Play

Fase 3

Práctica (80% mastery)

Fase 4

Paciencia + Perseverancia

El proyecto se divide en dos equipos sobre el mismo monolito modular: **Equipo 1** — **Gap Engine** produce el `GapReport`, **Equipo 2** (este repo) consume el reporte y opera las fases 1 a 4, además de la supervisión humana.

2 Arquitectura general

Patrón de módulo

Cada capacidad vive en `app/modules/<cap>/` con capas estrictas. Routers no tocan SQLAlchemy. Services no arman queries. Repositorios no levantan HTTPException.

router

HTTP, payloads

service

reglas + errores

repository

SQLAlchemy

models

tablas

schemas

Pydantic

IA híbrida + fallback

El código fija la estructura del plan (módulos por skill, 3 fases, 5 ejercicios A/B/C/D). El LLM solo rellena contenido. Si el LLM falla, cae a Mock sin romper la API.

Protocol + Factory permite cambiar Groq ↔ Gemini ↔ Mock por env var `PLAN_GENERATOR` sin tocar código.

3 Lo realizado

Avance del proyecto

43%

6 / 14 épicas

● Cerradas 6 ● Pendientes 8

Esfuerzo por épica (horas)

C1 users		4 h
C2 paths		6 h
C3 db+deploy		5 h
C4 IA		6 h
C5 attempts		3 h
C6 resources		3 h
Total		27 h

Módulos implementados (prefijo `/api`)

MÓDULO	ENDPOINTS CLAVE	CUBRE	ESTADO
users	<code>CRUD /users</code>	Padrón de actores (student, tutor, company_admin, admin)	✓
skills	<code>CRUD /skills</code> + seed	Catálogo de habilidades IT	✓
user_skills	<code>/students/{email}/skills</code>	Auto-reporte del estudiante	✓
job_descriptions	<code>CRUD /job-descriptions</code>	Ofertas + skills requeridas	✓
gap_analysis	upload + extracción + LIFO + disparo	Fase 0 (suplanta a Equipo 1)	✓
learning_paths	<code>POST/GET /learning-paths</code>	Fases 1, 2 y 3	✓
attempts	<code>POST/GET /attempts</code>	Fase 4 — Paciencia	✓

MÓDULO	ENDPOINTS CLAVE	CUBRE	ESTADO
resources	(consumido por learning_paths)	Recursos externos por skill+fase	✓

Épicas cerradas

- C1 · MVP backend FastAPI + módulo users · ~4 h**
Patrón de módulo en capas + CRUD de actores.
- C2 · learning_paths con MockPlanGenerator · ~6 h**
Estructura 5P, factory Protocol, contrato del GapReport.
- C3 · Persistencia PostgreSQL (Neon + Render) · ~5 h**
URL pública, auto-deploy desde main, pin Python 3.12.
- C4 · Generación con IA: Groq + Gemini + Mock · ~6 h**
Paralelización con ThreadPool, fallback automático, 5 ejercicios A/B/C/D por skill.
- C5 · attempts + Paciencia (4° P) · ~3 h**
Identificación compuesta, mastery 80%, feedback empático con LLM.
- C6 · resources con anti-alucinación de URLs · ~3 h**
Cache-aside por (skill, fase), whitelist de docs canónicas, búsquedas YouTube.

4

Cobertura del modelo 5P



Pasión

Play

Práctica

Paciencia

4 de 5 P implementadas. Perseverancia requiere módulos `sessions` + `student_metrics`.

P	IMPLEMENTACIÓN	ESTADO
Pasión	Fase del plan generada por LLM que conecta skill ↔ empresa concreta	✓
Play	Fase del plan con micro-contenido y recursos externos (sandbox, video)	✓
Práctica	5 ejercicios A/B/C/D, evaluación con umbral 80%	✓
Paciencia	Feedback empático generado por LLM sin revelar la respuesta	✓
Perseverancia	Streaks, progreso, métricas — requiere módulos <code>sessions</code> + <code>student_metrics</code>	Pendiente

5 Lo pendiente

Esfuerzo restante por épica (horas)

E1 Perseverancia		3–4 h
E2 Tutores		4–5 h
E3 Alertas		3–4 h
E4 Next-unit		2–3 h
E5 Validaciones		3 h
E6 Auth		9–12 h
E7 Eventos		2 h
E8 Calidad		4–6 h

Escala 0 → 12 h. Bandas muestran mínimo / máximo estimado.

Orden sugerido

1. **Épica 1** — Perseverancia (cierra las 5P). **3–4 h**

2. **Épica 2** — Tutores y asignaciones (HITL real). **4-5 h**
3. **Épica 3** — Alertas predictivas. **3-4 h**
4. **Épica 4** — Next-unit + desbloqueo progresivo. **2-3 h**

Las 4 juntas cierran el modelo de los documentos: **13-16 h** totales.

ÉPICA	POR QUÉ IMPORTA	ESFUERZO	PROGRESO
1 · Perseverancia + métricas	Cierra la 5° P del modelo	3-4 h	
2 · Tutores y asignaciones	HITL exigido por la consigna	4-5 h	
3 · Sistema de alertas	Análisis predictivo de abandono	3-4 h	
4 · Next-unit + desbloqueo	Mastery Learning real	2-3 h	
5 · Validaciones críticas	Entregables con juicio humano	3 h	
6 · Auth (mock + JWT)	Activa la matriz de accesos	9-12 h	
7 · Eventos hacia Equipo 1	Contrato append-only	2 h	
8 · Calidad técnica	Tests, CORS, Alembic	4-6 h	

6 Brechas respecto a la consigna

CONCEPTO DEL MODELO	BRECHA	IMPACTO
Grafo de conocimientos con prerequisites formales	Hoy las skills son lista plana, el orden lo decide el priority del GapReport	Medio
Modelo Pedagógico multi-agente (Estratega + Diseñador + Coach)	Un único LLM con prompts por fase	Medio
Estado afectivo en tiempo real (detección de frustración NLP)	No implementado	Alto
Contexto socioeconómico estructurado del alumno	No registrado en BD	Medio
Métricas psicométricas (Escala Grit, SDT)	No implementado	Medio

CONCEPTO DEL MODELO	BRECHA	IMPACTO
Plan de validación empírica con grupo control	No diseñado	Medio
Gobernanza de datos / consentimiento informado para menores	No implementado	Alto
Capacitación del tutor humano (onboarding del dashboard)	No diseñada	Medio

7 Decisiones técnicas

Auth diferida

Identificación por email mientras tanto. Matriz de permisos ya documentada para activar con un único `Depends`.

IA híbrida

Código fija la estructura; LLM solo rellena. Permite controlar la forma del output y caer a Mock sin romper.

Identificación compuesta

`learning_path_id + module_index + unit_index + exercise_index`, sin tabla exclusiva de ejercicios.

Gap analysis en 2 pasos

Partido por el timeout de 30s de Render free tier. Email es la clave funcional.

Anti-alucinación de URLs

Videos siempre como búsqueda en YouTube; docs canónicas solo desde dominios whitelisteados.

Fallback a Mock siempre activo

La API nunca rompe por culpa del LLM. `generator_used` en la respuesta marca cuál se usó.

8 Riesgos

RIESGO	MITIGACIÓN ACTUAL	MITIGACIÓN PENDIENTE
Timeout de Render free tier (30s)	Gap analysis partido en 2 pasos	Background task con <code>202 Accepted</code>
Cuota de Groq (14.4k RPD) y Gemini (1.5k RPD)	Fallback a Mock + cache de recursos	Cache de planes por (skill, role)
Sin tests automatizados	Manual vía Swagger + curls	pytest + httpx (Épica 8)
Sin Alembic — usa <code>create_all</code>	Funcional en MVP	Alembic antes del primer cambio con datos reales
Credenciales expuestas (Groq, Neon)	Variables en Render	Rotar antes de la presentación pública
Auth ausente	Endpoints públicos en MVP	Épica 6A (headers mock) desbloquea el resto

RIESGO	MITIGACIÓN ACTUAL	MITIGACIÓN PENDIENTE
PDFs escaneados (sin OCR)	Rechazo con 422 si el texto es muy corto	Documentado como limitación

9 Próximos pasos

1. **Épica 1 — Perseverancia.** Cierra las 5P. **3-4 h.**
2. **Épica 2 — Tutores.** Activa el HITL del modelo. **4-5 h.**
3. **Épica 3 — Alertas.** Completa el dashboard predictivo. **3-4 h.**
4. **Épica 4 — Next-unit.** Mastery Learning con bloqueo de avance. **2-3 h.**
5. **Épica 6A — Auth mock.** Hace efectiva la matriz de accesos. **3-4 h.**
6. **Épica 8 — Tests + CORS + Alembic.** Indispensable para defender el TP como producto. **4-6 h.**

10 Referencias

Documentación interna

- Cobertura 5P
- Contrato entre equipos
- Arquitectura backend
- Integración con IA
- Backlog completo
- Guía para el frontend
- Curls de prueba
- Matriz de accesos

Recursos externos

- Repositorio: github.com/marcelocassiovieira/capacitaya
- Producción: capacity-ar-ap.onrender.com

- Swagger: </docs>

Informe técnico (resumen)

Estado del proyecto Capacity AR al cierre de esta iteración: qué se construyó, qué falta, y riesgos identificados, a la luz de la consigna del modelo pedagógico definida en los documentos del TP.

1. Consigna y alcance

La consigna pide diseñar e implementar un **Sistema de Tutoría Inteligente (ITS)** que capacite a jóvenes de barrios vulnerables de Argentina para insertarse en el sector IT, articulado sobre el marco pedagógico de las **5 P** (Práctica, Paciencia, Perseverancia, Pasión, Play) con esquema **Human-in-the-Loop (HITL)**.

Los documentos de referencia definen el flujo en cinco fases:

- **Fase 0 — Diagnóstico de brecha:** evaluación cruzada entre perfil del estudiante y oferta laboral → `learning path` personalizado.
- **Fase 1 — Pasión:** anclaje motivacional, conectar cada skill con la empresa concreta.
- **Fase 2 — Play:** micro-lecciones, sandbox, exploración sin penalización.
- **Fase 3 — Práctica:** ejercicios adaptativos con umbral de dominio del 80%.
- **Fase 4 — Paciencia y Perseverancia:** refuerzo emocional transversal.

Además, los documentos exigen tres modelos arquitectónicos (Dominio, Estudiante, Pedagógico/multi-agente) y supervisión humana mediante un dashboard con análisis predictivo de riesgo de abandono.

El proyecto se divide en **dos equipos sobre el mismo monolito modular**:

- **Equipo 1 — Gap Engine:** produce el `GapReport` (Fase 0). Recibe los datos crudos del estudiante y de la oferta y genera el reporte de brecha.
- **Equipo 2 — Capacitación + Tutores** (este repo): consume el `GapReport` y produce el plan pedagógico, la ejecución del alumno, el feedback y la supervisión humana (Fases 1 a 4).

Contrato y división de responsabilidades documentados en `scope-equipos.md`.

2. Arquitectura general

Monolito modular en **Python 3.12 + FastAPI + SQLAlchemy 2.x síncrono**, persistido en **PostgreSQL serverless (Neon)** y deployado en **Render** con auto-deploy desde `main`.

Cada módulo en `app/modules/<capability>/` respeta la separación en capas:

Capa	Archivo	Responsabilidad
Router	<code>router.py</code>	Endpoints HTTP, validación de payloads

Capa	Archivo	Responsabilidad
Service	<code>service.py</code>	Reglas de negocio, orquestación, errores
Repository	<code>repository.py</code>	Único lugar con SQLAlchemy
Models	<code>models.py</code>	Modelos declarativos
Schemas	<code>schemas.py</code>	Contratos Pydantic

Los proveedores externos (LLMs, repositorios) se inyectan con el patrón **Protocol + Factory**, lo que permite cambiar de Groq a Gemini (o caer a Mock) sin tocar el resto del código.

Patrón de IA híbrida: el código determinístico arma la estructura del plan (módulos por skill, 3 fases por módulo, 5 ejercicios por fase de Práctica) y el LLM solo rellena título, contenido y opciones de ejercicios. Esto da control sobre la forma del output y permite el fallback automático a Mock si el LLM falla.

Documentación arquitectónica en [backend-onboarding.md](#), [ai-integration.md](#) y [training-module-design.md](#).

3. Lo realizado

3.1 Equipo 1 — Gap Engine

Estado: implementado por Equipo 2 en este repo como `gap_analysis` (porque el equipo 1 no entregó su versión todavía, este módulo lo supe).

Componente	Endpoint	Estado
Upload de 2 documentos (estudiante + oferta)	<code>POST /api/gap-analyses</code> (multipart)	✓
Extracción de texto PDF / DOCX / TXT	<code>gap_analysis/extractors.py</code>	✓
LLM (Groq) extrae <code>GapReport</code> validado con JSON Schema	<code>gap_analysis/service.py</code>	✓
Persistencia del gap pendiente (LIFO por email)	tabla <code>gap_analyses</code>	✓
Disparo del learning path por email	<code>POST /api/students/{email}/generate-learning-path</code>	✓

El flujo está partido en 2 pasos porque la combinación extracción + generación de plan supera el timeout de 30s de Render free tier. El segundo paso busca el último gap pendiente (`learning_path_id IS NULL`) y lo enlaza.

3.2 Equipo 2 — Capacitación

Módulos implementados

Módulo	Endpoints	Cubre
<code>users</code>	CRUD <code>/users</code>	Padrón de actores (student, tutor, company_admin, admin)
<code>skills</code>	CRUD <code>/skills</code> + seed	Catálogo de habilidades IT
<code>user_skills</code>	<code>/students/{email}/skills</code>	Auto-reporte de skills del estudiante
<code>job_descriptions</code>	CRUD <code>/job-descriptions</code>	Ofertas laborales con skills requeridas
<code>gap_analysis</code>	ver arriba	Fase 0 (suplantando Equipo 1)
<code>learning_paths</code>	POST/GET <code>/learning-paths</code>	Fases 1, 2 y 3 del modelo
<code>attempts</code>	POST/GET <code>/attempts</code>	Fase 4 (Paciencia, feedback empático)
<code>resources</code>	(consumido por learning_paths)	Recursos externos por skill+fase

Todos los endpoints están bajo prefijo `/api`.

Cobertura de las 5 P

P	Cómo se implementa	Estado
Pasión	Fase del plan generada por LLM con prompt que conecta la skill con la empresa concreta	✓
Play	Fase del plan con micro-contenido y recursos externos (sandbox, video)	✓
Práctica	Fase del plan con 5 ejercicios multiple_choice A/B/C/D, evaluación con umbral 80%	✓
Paciencia	Feedback empático generado por LLM cuando el alumno falla un ejercicio, sin revelar la respuesta	✓
Perseverancia	Visualización de progreso, streaks, métricas — requiere módulos <code>sessions</code> y <code>student_metrics</code>	✗

Mapeo detallado en [5p-coverage.md](#).

Modelo del Dominio, Estudiante y Pedagógico (según los docx)

- **Modelo del Dominio:** parcialmente implementado vía `skills` + dependencias declaradas en el prompt del LLM. **Falta el grafo formal con prerequisites** (hoy es una lista plana).
- **Modelo del Estudiante:** estado cognitivo está representado por `attempts` y `user_skills`. **Faltan:** rasgos de personalidad (Grit), estado afectivo en tiempo real, contexto socioeconómico estructurado.
- **Modelo Pedagógico (multi-agente):** hoy es un **único agente** (el LLM activo) con prompts especializados por fase. **No hay separación formal** Estratega / Diseñador / Coach.

Integración con IA

Cuatro puntos del sistema usan el LLM activo (Groq por default, Gemini como alternativa, Mock como fallback):

- 1. Extracción del `GapReport` desde texto.
- 2. Generación de units (contenido + 5 ejercicios) por skill.
- 3. Sugerencia de recursos externos por skill y fase, con guardrails anti-alucinación de URLs (whitelist + búsquedas YouTube).
- 4. Feedback empático cuando el alumno falla (Paciencia).

Si el LLM cae, el endpoint sigue funcionando con `generator_used: "mock"`.

Infraestructura y despliegue

- ☒ Producción pública: `https://capacity-ar-ap.onrender.com`.
- ☒ Auto-deploy desde `main`.
- ☒ Documentación de curls de prueba (`api-curls.md`).
- ☒ Guía para frontend (`frontend-guide.md`).

Backlog completado

Resumen de las 6 épicas cerradas:

Épica	Esfuerzo real
C1 — MVP backend FastAPI + módulo users	~4 h
C2 — <code>learning_paths</code> con MockPlanGenerator	~6 h
C3 — Persistencia en PostgreSQL (Neon + Render)	~5 h
C4 — Generación con IA: Groq + Gemini con fallback	~6 h
C5 — <code>attempts</code> con Paciencia (4° P)	~3 h
C6 — <code>resources</code> con anti-alucinación de URLs	~3 h

Total invertido: ~27 horas. Detalle en `backlog.md`.

4. Lo pendiente

4.1 Equipo 1 — Gap Engine "nativo"

Hoy `gap_analysis` está implementado por Equipo 2 como solución pragmática. La consigna original asume que Equipo 1 produce el `GapReport` con un pipeline propio (Google Forms + validación + scoring). Pendiente: definir si esa entrega independiente sucede o si `gap_analysis` queda como solución definitiva.

4.2 Equipo 2 — Backlog priorizado

Las 4 épicas que cierran el modelo 5P + HITL según los documentos del TP:

#	Épica	Por qué importa según la consigna	Esfuerzo
1	Perseverancia y métricas del alumno	Cierra la 5º P. Sin esto, "Perseverancia" queda como concepto sin implementación	3-4 h
2	Tutores y asignaciones	Implementa el HITL exigido por el modelo. Sin esto no hay supervisión humana real	4-5 h
3	Sistema de alertas	El dashboard del tutor con análisis predictivo de abandono es un requisito explícito	3-4 h
4	Next-unit logic y desbloqueo progresivo	Mastery Learning de Bloom: bloquear avance hasta dominar la skill anterior	2-3 h

Total mínimo para cumplir con el modelo de la consigna: **13-16 horas**.

4.3 Otras épicas planificadas

#	Épica	Esfuerzo
5	Validaciones críticas (entregables con ojo humano)	3 h
6	Auth (Fase 6A mock + Fase 6B JWT)	9-12 h
7	Eventos hacia Equipo 1 (contrato append-only)	2 h
8	Calidad técnica (tests, CORS, Alembic)	4-6 h

Detalle, user stories, endpoints y criterios de aceptación de cada épica en [backlog.md](#).

4.4 Brechas conceptuales respecto a la consigna

Punto de la consigna	Brecha actual
Grafo de conocimientos con prerequisites formales	Skills hoy son lista plana; el orden lo decide el priority del GapReport
Modelo Pedagógico multi-agente (Estratega + Diseñador + Coach)	Hoy es un único LLM con prompts por fase
Estado afectivo en tiempo real (detección de frustración vía NLP)	No implementado
Contexto socioeconómico estructurado del alumno	No registrado en BD
Métricas psicométricas (Escala de Grit, SDT)	No implementado
Plan de validación empírica (piloto con grupo control)	No diseñado todavía
Gobernanza de datos y consentimiento informado	No implementado (relevante porque trabajamos con menores en barrios vulnerables)

Punto de la consigna	Brecha actual
Capacitación del tutor humano (onboarding del dashboard)	No diseñada

Estas brechas no son bloqueantes para el TP, pero **deben mencionarse en la presentación** porque están explícitas en los documentos del modelo.

5. Decisiones técnicas relevantes

- **Auth diferida** intencionalmente para no bloquear el resto. Identificación por email mientras tanto. Matriz de permisos ya documentada en [access-matrix.md](#).
- **IA híbrida** (código fija la estructura, LLM solo rellena contenido) en vez de pipeline 100% LLM. Permite controlar la forma del output y caer a Mock sin romper.
- **Identificación compuesta de ejercicios** (`learning_path_id + module_index + unit_index + exercise_index`) en vez de tabla propia. Trade-off documentado en [training-module-design.md](#).
- **Gap analysis partido en 2 pasos** por el timeout de 30s de Render free tier.
- **Anti-alucinación de URLs en recursos**: para videos siempre URLs de búsqueda en YouTube; para docs canónicas, whitelist de dominios.
- **No remover el fallback a Mock**: la API nunca debe romperse por culpa del LLM.

6. Riesgos

Riesgo	Mitigación actual	Mitigación pendiente
Timeout de Render free tier (30s)	Gap analysis partido en 2 pasos	Mover generación de plan a background task con <code>202 Accepted</code>
Cuota de Groq (~14.4k RPD) y Gemini (1.5k RPD)	Fallback a Mock + cache de recursos	Cache de planes por (skill, target_role) si crece el tráfico
Sin tests automatizados todavía	Manual via Swagger + curls	Épica 8 (pytest + httpx)
Sin Alembic (uso de <code>create_all</code>)	Funcional en MVP	Alembic antes del primer cambio de schema con datos reales
Credenciales en variables de entorno de Render	OK	Rotar las claves Groq + Neon antes de la presentación pública
Auth ausente	Endpoints públicos en MVP	Épica 6A (headers mock) desbloquea el resto del backlog
PDFs escaneados (sin OCR)	Rechazo con <code>422</code> (texto muy corto)	Documentado como limitación

7. Próximos pasos recomendados

Para la próxima iteración, ordenados por impacto en la consigna:

1. **Épica 1 (Perseverancia)**: cierra las 5 P. 3-4 h.
 2. **Épica 2 (Tutores)**: activa el HITL exigido por la consigna. 4-5 h.
 3. **Épica 3 (Alertas)**: completa el dashboard predictivo. 3-4 h.
 4. **Épica 4 (Next-unit logic)**: Mastery Learning real con bloqueo de avance. 2-3 h.
 5. **Épica 6A (Auth mock)**: hace efectiva la matriz de accesos documentada. 3-4 h.
 6. **Épica 8 (Tests + CORS + Alembic)**: indispensable para defender el TP como producto, no como prototipo. 4-6 h.
-

8. Referencias

- Modelo pedagógico y arquitectura: [Modelo IA para Capacitación IT.docx](#), [Scrum_Trabajo_Practico.docx](#), [Modelo Capacitacion con IA 2.docx](#).
- Cobertura 5P: [5p-coverage.md](#).
- Contrato entre equipos: [scope-equipos.md](#).
- Arquitectura backend: [backend-onboarding.md](#).
- Integración con IA: [ai-integration.md](#).
- Backlog completo: [backlog.md](#).
- Guía para el frontend: [frontend-guide.md](#).
- Curls de prueba: [api-curls.md](#).
- Matriz de accesos: [access-matrix.md](#).
- Repositorio: <https://github.com/marcelocassiovieira/capacitaya>
- Producción: <https://capacity-ar-ap.onrender.com>

Capacity AR

scope-equipos.md

Scope entre equipos

Documento de acuerdo entre equipos sobre quien hace que dentro del monolito modular Capacity AR.

Contexto

El sistema se divide en 2 equipos que trabajan sobre el mismo backend modular. Una sola API, una sola base de datos, modulos internos. No es microservicios.

Mapa De Alto Nivel

```
EMPRESA + ESTUDIANTES
|
v
+-----+
| EQUIPO 1 |
| Gap + Metricas globales |
+-----+
| GapReport JSON
v
+-----+
| EQUIPO 2 (nosotros) |
| Capacitacion + Tutores |
+-----+
```

Flujo unidireccional. Equipo 1 produce el insumo. Equipo 2 lo consume y ejecuta el proceso pedagogico.

Equipo 1 - Gap + Metricas Globales

Responsabilidades

1. Onboarding de estudiantes - Recibir datos del form de Google (webhook o Apps Script). - Crear el student_profile en la base.
2. Gestion de empresas y ofertas - Form web para que la empresa cargue sus datos. - Form web para crear y editar posiciones con habilidades requeridas.
3. Gap Engine - Recibir student + company + target_role. - Calcular gap por habilidad, status (READY/NEEDS_WORK/MISSING), readiness_score. - Persistir el GapReport en JSON. - Especificacion detallada en [gap-engine-mvp.md](#).

4. Metricas globales (dashboard admin) - Tasa de completitud del path por cohorte. - Tiempo promedio hasta dominio del 80%. - Tasa de desercion. - Distribucion de readiness final. - Tasa de insercion laboral (largo plazo). - Para MVP solo lo ve el admin. La empresa no accede a metricas.

Sub-modulos

```
app/modules/  
  students/  
  companies/  
  gap_analysis/  
  global_metrics/
```

Endpoints principales

```
POST    /students                (desde webhook form Google)  
GET     /students/{id}  
POST    /companies  
GET     /companies  
POST    /companies/{id}/positions  
POST    /api/v1/gap-analyses  
GET     /api/v1/gap-analyses/{id}  
GET     /admin/metrics/global
```

Output al Equipo 2

GapReport JSON con el formato definido en `gap-engine-mvp.md`.

Input desde el Equipo 2

Lectura de tablas compartidas (`attempts` , `sessions` , `events`) para sus agregados.

Equipo 2 - Capacitacion + Tutores

Responsabilidades

Parte A - Capacitacion (motor 5P)

1. Generacion del learning path - Consumir el GapReport JSON. - Crear modulos (1 por habilidad faltante), unidades (5P), ejercicios.
2. Contenido adaptativo con IA - Integracion con Google Gemini. - Generar explicaciones, ejercicios y feedback contextualizados al perfil e intereses del estudiante.
3. Ciclo pedagogico 5P - Pasion: anclaje motivacional con la empresa elegida. - Play: sandbox, micro-lecciones de 5 a 10 minutos. - Practica: ejercicios adaptativos, umbral de dominio 80%. - Paciencia y Perseverancia: refuerzo emocional transversal al ciclo.

4. Registro de actividad - Sesiones, intentos, dominio por skill. - Emision de eventos que consume el Equipo 1 para metricas globales.
5. Metricas individuales - Progreso del alumno (vista motivacional). - Datos de seguimiento del tutor (frecuencia, riesgo, etc.).

Parte B - Tutores

6. Padron y asignacion - CRUD de tutores. - Asignar tutor a un estudiante (manual o automatica - a definir).
7. Sistema de alertas internas - Detectar senales (low_perseverance, abandon_risk, lost_passion, anomaly_patience). - Notificar al tutor asignado.
8. Validaciones criticas - Endpoints para que el estudiante suba entregables. - Endpoints para que el tutor valide presentaciones o codigo.
9. Dashboard del tutor - Listado de asignados. - Alertas activas. - Estado de cada estudiante.

Sub-modulos

```
app/modules/  
  learning_paths/  
  learning_content/  
  attempts/  
  sessions/  
  tutors/  
  tutor_assignments/  
  alerts/  
  validations/  
  student_metrics/
```

Endpoints principales

```
POST   /learning-paths  
GET    /learning-paths/{id}  
GET    /learning-paths/{id}/next-unit  
POST   /attempts  
GET    /students/{id}/progress  
GET    /tutors  
POST   /tutors  
POST   /tutor-assignments  
GET    /tutors/{id}/dashboard  
GET    /alerts  
POST   /validations
```

Modulos Compartidos

```
app/modules/  
  users/      (ya implementado)
```

auth/	(post-MVP - login JWT)
shared/	(tipos comunes, db, utilidades)

Pieza	Estado	Quien la mantiene
users/	Implementado	Cualquiera, no se toca mucho
auth/	Post-MVP	Por decidir
students , companies (modelos)	Pendiente	Equipo 1
tutors , learning_paths (modelos)	Pendiente	Equipo 2

Flujo End To End

1. EMPRESA carga oferta (Eq1)
2. ESTUDIANTE completa form Google (Eq1)
3. ESTUDIANTE elige posicion (Eq1)
4. SE GENERA EL GAP REPORT (Eq1)
5. SE GENERA EL LEARNING PATH (Eq2)
6. ESTUDIANTE APRENDE (5P + IA) (Eq2)
7. SISTEMA DETECTA RIESGO (Eq2)
8. TUTOR INTERVIENE SI HACE FALTA (Eq2)
9. ADMIN/EMPRESA VEN METRICAS (Eq1)
10. ESTUDIANTE LISTO PARA POSTULAR

Contratos Entre Equipos

Eq1 -> Eq2: GapReport

Definido en `gap-engine-mvp.md` . Formato JSON estable.

Eq2 -> Eq1: Tablas compartidas

El Equipo 1 puede leer las siguientes tablas para sus metricas agregadas:

- sessions
- attempts
- events (tabla append-only con eventos del flujo pedagogico)
- skill_mastery

Eventos a definir

Pendiente acordar el contrato exacto de eventos que emite el Equipo 2. Lista inicial sugerida:

```
session_started
session_ended
exercise_attempted
```

```
exercise_completed
unit_completed
module_completed
path_completed
mastery_achieved (cuando llega al 80% de una skill)
alert_raised
tutor_intervened
```

Decisiones Pendientes

1. Contrato exacto de eventos Eq2 -> Eq1.
2. Tipo de respuesta del estudiante en ejercicios: texto libre, multiple opcion o codigo.
3. Orden de las fases 5P: fijo por habilidad o decidido por la IA.
4. Algoritmo de asignacion de tutor: manual, round-robin o por especialidad.
5. Responsable del modulo `auth/` cuando se sume.
6. Responsable del frontend.

Resumen

Aspecto	Equipo 1	Equipo 2
Entrada al sistema	Empresa, Estudiante	-
Analisis inicial	Gap Report	-
Motor pedagogico	-	5P + IA Gemini
Acompañamiento humano	-	Tutores + Alertas
Metricas	Globales (admin/empresa)	Individuales (alumno/tutor)
Punto de integracion	Entrega GapReport	Consume GapReport

Backend onboarding

Este backend arranca como monolito modular. La unidad de despliegue es una sola API, pero el código se organiza por módulos de negocio para que después sea posible extraer partes si realmente aparece la necesidad.

Dominio inicial

De los documentos del proyecto, el sistema apunta a una plataforma de aprendizaje adaptativo para reducir la brecha entre formación secundaria y mercado laboral IT.

Actores iniciales:

- `student`: estudiante que realiza diagnóstico, learning path, prácticas y evaluaciones.
- `tutor`: humano que monitorea progreso, riesgo de abandono y evaluaciones críticas.
- `company_admin`: referente de empresa/oferta laboral que puede necesitar ver avances.
- `admin`: operación interna de la plataforma.

Por eso el primer módulo es `users`: no resuelve todo el dominio, pero permite registrar los actores sobre los que después cuelgan onboarding, diagnósticos, cursos, ofertas y seguimiento.

Flujo de trabajo local

Activar entorno:

```
source .venv/bin/activate
```

Levantar API:

```
uvicorn app.main:app --reload
```

Abrir documentación Swagger:

```
http://localhost:8000/docs
```

Crear usuario:

```
curl -X POST http://localhost:8000/users \
  -H "Content-Type: application/json" \
  -d
```

```
{ "first_name": "Ana", "last_name": "Perez", "email": "ana@example.com", "role": "student" }
```

Listar usuarios:

```
curl http://localhost:8000/users
```

Dónde poner cada cosa

`router.py` recibe HTTP, valida payloads y delega en servicios. Mantiene los endpoints finos y evita mezclar reglas de negocio con transporte.

`service.py` contiene reglas de negocio y decisiones de error. Por ejemplo: no permitir emails duplicados.

`repository.py` encapsula SQLAlchemy. Si mañana cambia una query, idealmente no toca el router.

`models.py` define tablas y columnas SQLAlchemy.

`schemas.py` define contratos Pydantic para requests y responses.

Decisiones MVP

- SQLAlchemy síncrono: más simple para empezar; FastAPI lo soporta bien.
- `create_all` al iniciar: aceptable para MVP sin Alembic.
- Sin Docker local: se puede correr directo con Python y PostgreSQL.
- Sin auth todavía: primero cerrar CRUD y flujos de dominio.
- Sin microservicios: módulos internos, una sola base y un solo deploy.

Próximo módulo sugerido

Después de `users`, el siguiente corte natural es `learning_profiles` u `onboarding`, con datos mínimos del estudiante:

- puesto objetivo
- intereses
- disponibilidad horaria
- nivel técnico inicial
- habilidades declaradas

Eso conecta directamente con la Fase 0/Fase 1 descrita en los documentos: diagnóstico de brecha y anclaje motivacional.

Integración con IA

Cómo el sistema usa LLMs para generar contenido pedagógico.

Lo esencial

- No entrenamos ni hicimos fine-tuning. Usamos un LLM pre-entrenado vía API.
- Dos proveedores soportados:** Groq (Llama 3.3 70B, default) y Google Gemini. Cambiás con la env var `PLAN_GENERATOR`.
- La estructura del plan (módulos, fases, orden) la decide código determinístico. Solo el contenido textual y los ejercicios los genera el LLM.
- Si el LLM falla, hay fallback automático a `MockPlanGenerator`. El campo `generator_used` en cada plan indica qué se usó (`groq` / `gemini` / `mock`).

Dónde se usa el LLM

Punto	Archivo	Qué pide
Generar units del plan	<code>learning_paths/plan_generator/_base_llm.py</code>	title + content + 5 ejercicios multiple_choice
Sugerir recursos de estudio	<code>resources/suggester.py</code>	videos, guías, sandboxes — cacheados por (skill, fase)
Feedback empático en fallo	<code>attempts/feedback.py</code>	mensaje de Paciencia sin revelar la respuesta

Qué decide el código y qué decide la IA

Decisión	Quién
Skills que entran al plan, orden, prioridad	Código
Cantidad de units por módulo (3), duración (10/15/30 min)	Código
Validación de respuesta del alumno, cálculo de mastery	Código
Title, content y ejercicios de cada unit	LLM
Recursos sugeridos	LLM (con guardrails anti-alucinación de URLs)
Mensaje de feedback ante error	LLM

La estrategia híbrida nos da consistencia (la estructura no varía aunque el LLM se equivoque), permite auditar las reglas de negocio en el código, y reduce costo (menos llamadas).

Proveedores

Aspecto	Groq	Gemini
Modelo usado	llama-3.3-70b-versatile	gemini-2.5-flash-lite
Free tier RPD	14.400	1.500
Latencia típica por call	1-3 s	3-7 s
API key	https://console.groq.com/keys	https://aistudio.google.com/apikey

Para cambiar entre proveedores: editar `PLAN_GENERATOR` en Render → Environment. Save → redeploya solo. No hay que tocar código.

Para cambiar el modelo dentro del mismo proveedor: editar `GROQ_MODEL` o `GEMINI_MODEL`.

Configuración

Variable	Valores
<code>PLAN_GENERATOR</code>	<code>mock</code> / <code>groq</code> / <code>gemini</code>
<code>GROQ_API_KEY</code> , <code>GROQ_MODEL</code>	requeridas si <code>PLAN_GENERATOR=groq</code>
<code>GEMINI_API_KEY</code> , <code>GEMINI_MODEL</code>	requeridas si <code>PLAN_GENERATOR=gemini</code>

Las variables del proveedor inactivo se pueden dejar configuradas — el código las ignora.

Manejo de fallas

Todas las fallas del LLM (key faltante, 429 por cuota, 404 por modelo, JSON inválido) caen al fallback Mock. El plan se persiste igual con `generator_used: "mock"`. Si el suggerter de resources falla, las units quedan con `resources: []`. Si el feedback de attempts falla, se devuelve un texto estático.

La única falla que el sistema no oculta es el timeout de Render (30s en free tier). En ese caso el cliente recibe error HTTP y el plan no se persiste.

Para defensa del TP

No entrenamos ni hicimos fine-tuning. Usamos LLMs pre-entrenados de propósito general (Groq Llama 3.3 70B como default, Gemini como alternativa) y aplicamos prompt engineering para alinear la salida al modelo pedagógico 5P. La inteligencia del dominio vive en nuestros prompts (`prompts.py`) y en la estructura híbrida que mantiene las reglas pedagógicas en código. El LLM aporta capacidad de generación de texto natural y razonamiento general.

Reglas que mantener

1. **No mover lógica de negocio a los prompts.** El prompt define tono y formato; las reglas viven en `_base_llm.py` y `service.py`.
2. **No remover el fallback a Mock.** Es lo que garantiza disponibilidad sin LLM.
3. **No cachear respuestas del LLM por estudiante.** Cada plan es personalizado a sus intereses. La excepción es `resources`, que cachea por (skill, fase) porque es independiente del estudiante.

Cobertura del modelo 5P

Mapea las cinco "P" del modelo pedagógico de Capacity AR a piezas concretas del backend.

Los documentos del modelo dividen las P en dos grupos:

- **3 P que generan contenido del plan** — son unidades del plan generado.
- **2 P transversales en runtime** — son comportamientos del sistema durante el uso del plan, no fases del plan.

Cita textual de `docs/training-module-design.md`:

"Paciencia y Perseverancia operan transversalmente (no son fases separadas)."

Mapa

P	Tipo	Estado	Dónde
Pasión	Contenido del plan	✓ Implementada	<code>learning_paths/plan_generator/prompts.py</code> (fase <code>pasion</code>)
Play	Contenido del plan	✓ Implementada	<code>prompts.py</code> (fase <code>play</code>)
Práctica	Contenido del plan	✓ Implementada	<code>prompts.py</code> (fase <code>practica</code>)
Paciencia	Transversal	✓ Implementada	<code>attempts/feedback.py</code> (feedback empático al fallar)
Perseverancia	Transversal	⚠ Pendiente	irá en <code>student_metrics/</code> + <code>alerts/</code>

Cómo verificar en producción

- **Pasión:** `modules[*].units[0].content` de cualquier plan generado menciona el nombre del estudiante, sus intereses y la empresa objetivo.
- **Play:** `modules[*].units[1].content` con tono lúdico e invitación a experimentar. `exercises: []`.
- **Práctica:** `modules[*].units[2].exercises` con 5 multiple_choice (A/B/C/D, `expected_answer` letra).
- **Paciencia:** `POST /attempts` con respuesta incorrecta devuelve `ai_feedback` con mensaje empático que **no revela la respuesta correcta**.
- **Perseverancia:** aún no expuesta. Cuando esté `GET /students/{email}/progress`, ahí va.

Lo que falta de Paciencia y Perseverancia

- **Detección de patrones agregados de frustración** (varios fallos seguidos, lenguaje agresivo): módulo `alerts/` (no implementado).
- **Visualización del progreso del alumno** (% completado, streak, hitos): módulo `student_metrics/` (no implementado).
- Diseño detallado de ambos en `training-module-design.md`.

Capacity AR

gap-engine-mvp.md

Gap engine MVP

Proposito

El primer modulo central del sistema sera el Gap Engine.

Su responsabilidad es comparar el perfil actual de un estudiante contra los requerimientos de un puesto objetivo y generar un informe de brecha accionable.

```
student profile + target role/company requirements
-> Gap Engine
-> gap report JSON
-> Training Module
-> learning path
```

El Gap Engine pertenece a nuestro sistema. Analiza la informacion recibida, calcula la brecha y devuelve un JSON estructurado con el informe.

Este modulo no genera el plan de capacitacion. Solo responde que conocimientos, habilidades o competencias le faltan al estudiante para acercarse al puesto objetivo.

El plan de capacitacion sera responsabilidad de otro modulo interno posterior. Ese modulo consumira el JSON generado por el Gap Engine y lo transformara en modulos, unidades y ejercicios.

Alcance Del MVP

El MVP del Gap Engine debe permitir:

- Recibir datos del estudiante.
- Recibir el puesto objetivo.
- Recibir los requerimientos de habilidades del puesto.
- Comparar nivel actual vs nivel requerido.
- Calcular brecha por habilidad.
- Calcular un score general de preparacion.
- Persistir el informe.
- Consultar el informe generado.

No incluye por ahora:

- IA generativa.
- Cola de mensajeria.
- Procesamiento asincronico.
- Historial avanzado de evaluaciones.
- Modelo complejo de skills en tablas normalizadas.

- Motor semantico de equivalencias entre habilidades.
- Generacion del learning path.

Modulo Propuesto

```
app/modules/gap_analysis/  
  router.py  
  service.py  
  repository.py  
  models.py  
  schemas.py  
  engine.py
```

Responsabilidades:

- `router.py` : endpoints HTTP.
- `service.py` : orquestacion del caso de uso.
- `repository.py` : persistencia.
- `models.py` : tabla principal del informe.
- `schemas.py` : contratos de request/response.
- `engine.py` : algoritmo deterministico de calculo de brecha.

Endpoints MVP

Crear Analisis De Brecha

```
POST /api/v1/gap-analyses
```

Request:

```
{  
  "student": {  
    "name": "Ana Perez",  
    "email": "ana@example.com",  
    "skills": [  
      {  
        "name": "Git",  
        "level": 1  
      },  
      {  
        "name": "SQL",  
        "level": 2  
      }  
    ],  
    "interests": ["logistica", "backend"]  
  },  
  "company": {  
    "name": "Empresa Logistica SA"  
  },  
  "target_role": {
```

```
    "title": "Backend Developer Junior",
    "required_skills": [
      {
        "name": "Git",
        "level": 3,
        "priority": "HIGH"
      },
      {
        "name": "SQL",
        "level": 3,
        "priority": "HIGH"
      },
      {
        "name": "HTTP APIs",
        "level": 2,
        "priority": "MEDIUM"
      }
    ]
  }
}
```

Response:

```
{
  "id": 1,
  "student": {
    "name": "Ana Perez",
    "email": "ana@example.com"
  },
  "company": {
    "name": "Empresa Logistica SA"
  },
  "target_role": {
    "title": "Backend Developer Junior"
  },
  "summary": "Ana Perez necesita reforzar Git, SQL y HTTP APIs para acercarse al puesto Backend Developer Junior.",
  "readiness_score": 45,
  "skills": [
    {
      "name": "Git",
      "current_level": 1,
      "required_level": 3,
      "gap_level": 2,
      "priority": "HIGH",
      "status": "MISSING"
    },
    {
      "name": "SQL",
      "current_level": 2,
      "required_level": 3,
      "gap_level": 1,
      "priority": "HIGH",
      "status": "NEEDS_WORK"
    }
  ]
}
```

```
{
  "name": "HTTP APIs",
  "current_level": 0,
  "required_level": 2,
  "gap_level": 2,
  "priority": "MEDIUM",
  "status": "MISSING"
},
{
  "created_at": "2026-05-12T15:30:00"
}
```

Consultar Analisis

```
GET /api/v1/gap-analyses/{id}
```

Devuelve el informe previamente generado.

Reglas De Calculo

Nivel De Habilidad

Para el MVP, cada habilidad se representa con un nivel numerico simple:

```
0 = no declarado / no sabe
1 = basico
2 = inicial operativo
3 = requerido para junior
4 = intermedio
5 = avanzado
```

Esta escala puede cambiar, pero es suficiente para validar el flujo.

Gap Por Habilidad

```
gap_level = required_level - current_level
```

Si el estudiante no declara una habilidad requerida, se asume:

```
current_level = 0
```

Estado Por Habilidad

```
READY      -> gap_level <= 0
NEEDS_WORK -> gap_level == 1
MISSING     -> gap_level >= 2
```

Prioridad

Prioridades aceptadas:

```
HIGH  
MEDIUM  
LOW
```

Pesos iniciales:

```
HIGH    = 3  
MEDIUM = 2  
LOW     = 1
```

Readiness Score

El score mide que tan cerca esta el estudiante del puesto objetivo.

Formula MVP:

```
skill_score = min(current_level, required_level) / required_level  
weighted_score = skill_score * priority_weight  
readiness_score = sum(weighted_score) / sum(priority_weight) * 100
```

Resultado:

```
0    = no cubre ningun requerimiento  
100 = cubre todos los requerimientos
```

Persistencia MVP

Tabla:

```
gap_analyses
```

Campos:

```
id  
student_name  
student_email  
company_name  
target_role_title  
readiness_score  
summary  
input_json  
result_json
```

```
created_at
updated_at
```

Para el MVP se guardan `input_json` y `result_json` como JSON.

Motivo: evita modelar prematuramente tablas de skills, roles, empresas, evaluaciones y equivalencias. Cuando el flujo este validado, se puede normalizar.

Ejemplo De Interpretacion

Si el puesto requiere:

```
Git nivel 3 HIGH
SQL nivel 3 HIGH
HTTP APIs nivel 2 MEDIUM
```

Y el estudiante tiene:

```
Git nivel 1
SQL nivel 2
HTTP APIs no declarado
```

Entonces:

```
Git      -> gap 2 -> MISSING
SQL      -> gap 1 -> NEEDS_WORK
HTTP APIs -> gap 2 -> MISSING
```

El informe debería indicar que Git y HTTP APIs son las brechas principales, y SQL necesita refuerzo.

Relacion Con El Modulo De Capacitacion

El Gap Engine produce el insumo principal para el modulo de capacitacion.

Ambos modulos son parte del mismo backend modular:

```
app/modules/gap_analysis/
    -> genera GapReport JSON

app/modules/learning_paths/
    -> consume GapReport JSON
    -> genera plan de capacitacion
```

Flujo posterior:

```
GapAnalysis.result_json
    -> Learning Path Generator
    -> Training Plan
```

El modulo de capacitacion tomara:

- habilidades faltantes,
- prioridad,
- nivel actual,
- nivel requerido,
- intereses del estudiante,
- puesto objetivo,
- empresa,

y generara:

- modulos,
- unidades,
- ejercicios,
- orden sugerido,
- tutor asignado,
- criterios de avance.

Decisiones De Simplicidad

Para maximizar time to market:

- El calculo sera deterministico.
- No habra IA en el primer corte.
- No habra procesamiento asincronico.
- No se usara cola.
- No se normalizaran habilidades todavia.
- Se persistira input y output completo para trazabilidad.
- No se implementara login en este primer corte tecnico.

La autenticacion queda diferida para antes de presentar el MVP. Mientras tanto, los endpoints se consideran publicos solo para desarrollo local o demo controlada.

Antes de presentar el MVP se debera agregar:

- registro/login basico,
- JWT,
- proteccion de endpoints,
- roles minimos para estudiante, tutor, empresa y admin.

Esto permite demostrar rapido el valor central:

dados un estudiante y un puesto objetivo,
el sistema puede explicar claramente la brecha.

Proximo Paso De Implementacion

Implementar:

```
POST /api/v1/gap-analyses
GET /api/v1/gap-analyses/{id}
```

con:

- validacion Pydantic,
- calculo en `engine.py`,
- persistencia en SQLite local/PostgreSQL Render via SQLAlchemy,
- respuesta JSON lista para ser consumida por el futuro modulo de capacitacion.

Capacity AR

training-module-design.md

Diseño del módulo de entrenamiento

Diseño técnico del módulo de capacitación del Equipo 2. Cubre el motor 5P, la integración con IA, el sistema de tutores, alertas y métricas individuales.

Vision General

El módulo de capacitación es la pieza central del sistema. Recibe un GapReport del Equipo 1 y construye una experiencia de aprendizaje adaptativa para el estudiante, basada en el marco pedagógico de las 5 P y supervisada por un tutor humano.

GapReport JSON -> Learning Path -> 5P + IA + Tutor -> Estudiante listo

Entrada

Definida por el Equipo 1 en `gap-engine-mvp.md`. Resumen:

```
{
  "student": {...},
  "company": {...},
  "target_role": {...},
  "readiness_score": 45,
  "skills": [
    { "name": "Git", "gap_level": 2, "priority": "HIGH", "status": "MISSING", ... },
    { "name": "SQL", "gap_level": 1, "priority": "HIGH", "status": "NEEDS_WORK", ... }
  ]
}
```

Salida

Un learning path con:

- Módulos (1 por habilidad faltante).
- Unidades dentro de cada módulo, alineadas a las fases 5P.
- Ejercicios generados o validados con IA.
- Orden sugerido según prioridad y dependencias.
- Tutor asignado.
- Criterios de avance (umbral 80%).

Marco Pedagogico - Las 5 P

Fase	Que hace	Salida
Pasion	Anclaje motivacional, conecta habilidad con la empresa elegida	Mensaje contextualizado
Play	Sandbox, micro-leccion 5-10 min, exploracion sin penalizacion	Unidad interactiva
Practica	Ejercicios adaptativos, umbral de dominio 80%	Evaluacion formativa
Paciencia	Detecta frustracion, normaliza el error	Mensaje empatico
Perseverancia	Visualiza progreso, micro-logros	Hito visible

Paciencia y Perseverancia operan transversalmente (no son fases separadas).

Sub-modulos

```
app/modules/  
  learning_paths/      # path completo del estudiante  
  learning_content/    # unidades, ejercicios, integracion Gemini  
  attempts/           # intentos + logica dominio 80%  
  sessions/           # registro de sesiones  
  tutors/             # padron de tutores  
  tutor_assignments/   # asignacion estudiante-tutor  
  alerts/             # alertas internas  
  validations/        # entregables que requieren tutor humano  
  student_metrics/    # metricas individuales (alumno y tutor)
```

Cada sub-modulo respeta el patron de `users/` : `router.py` , `service.py` , `repository.py` , `models.py` , `schemas.py` .

Modelo De Datos Propuesto

learning_paths

```
id  
gap_analysis_id      (FK a gap_analyses del Eq1)  
student_id          (FK)  
status              (ACTIVE | COMPLETED | PAUSED | ABANDONED)  
readiness_target    (score objetivo, default 80)  
created_at  
updated_at
```

learning_modules

Un modulo por habilidad faltante del gap.

```
id
learning_path_id      (FK)
skill_name            (Git, SQL, etc.)
priority              (HIGH | MEDIUM | LOW)
order_index
mastery_threshold     (default 0.80)
current_mastery       (0.0 a 1.0)
status                (PENDING | IN_PROGRESS | MASTERED)
```

learning_units

Lecciones dentro de un modulo, alineadas a 5P.

```
id
module_id             (FK)
phase                 (PASION | PLAY | PRACTICA)
title
content_json          (texto, ejemplos, generado por IA)
order_index
estimated_minutes
```

exercises

```
id
unit_id               (FK)
prompt
type                  (MULTIPLE_CHOICE | TEXT | CODE) - a definir con equipo
expected_answer
difficulty             (1 a 5)
```

attempts

```
id
exercise_id           (FK)
student_id            (FK)
unit_id               (FK)
skill_name
answer
is_correct
score                 (0.0 a 1.0)
retries
time_spent_seconds
ai_feedback           (generado por Gemini)
created_at
```

sessions

```
id
student_id          (FK)
learning_path_id    (FK)
started_at
ended_at
duration_seconds
```

skill_mastery

Estado actual de dominio del estudiante por habilidad.

```
id
student_id          (FK)
skill_name
current_level       (0 a 5)
target_level
mastery_percentage  (0.0 a 1.0)
last_practiced_at
```

events

Tabla append-only que sirve al Equipo 1 para sus metricas globales.

```
id
student_id
type                (session_started | exercise_completed | mastery_achieved |
etc.)
payload_json
created_at
```

tutors

```
id
user_id             (FK a users)
specialties         (lista de skills, ej: ["Git", "Python"])
max_students
available
```

tutor_assignments

```
id
tutor_id            (FK)
student_id          (FK)
learning_path_id    (FK)
assigned_at
unassigned_at       (nullable)
```

alerts

```
id
learning_path_id      (FK)
student_id            (FK)
type                  (LOW_PERSEVERANCE | ABANDON_RISK | LOST_PASSION |
ANOMALY_PATIENCE)
level                 (1 a 3)
payload_json          (variables que dispararon la alerta)
resolved
resolved_by_tutor_id  (FK nullable)
created_at
resolved_at           (nullable)
```

validations

Entregables que requieren ojo humano.

```
id
learning_path_id      (FK)
student_id            (FK)
unit_id               (FK)
type                  (PRESENTATION | CODE_REVIEW | PROJECT)
content_url           (link al entregable)
status                (PENDING | APPROVED | REJECTED | NEEDS_REVISION)
tutor_feedback
validated_by_tutor_id (FK nullable)
submitted_at
validated_at          (nullable)
```

Endpoints

Learning Path

```
POST  /learning-paths      (crea desde un GapReport)
GET    /learning-paths/{id}
GET    /students/{id}/learning-path (path activo del estudiante)
GET    /learning-paths/{id}/next-unit (que toca hacer ahora)
PATCH /learning-paths/{id} (tutor o admin pueden ajustar)
```

Contenido y ejercicios

```
GET    /units/{id}
GET    /units/{id}/exercises
POST   /attempts           (registra intento, IA evalua)
GET    /attempts/{id}
GET    /students/{id}/attempts (historial)
```

Sesiones

```
POST /sessions/start
POST /sessions/{id}/end
GET /students/{id}/sessions
```

Tutores

```
POST /tutors
GET /tutors
GET /tutors/{id}
PATCH /tutors/{id}
POST /tutor-assignments (asigna tutor a estudiante)
GET /tutors/{id}/dashboard (panel del tutor)
GET /tutors/{id}/students (asignados)
```

Alertas

```
GET /alerts (filtrable por tutor, estudiante, estado)
GET /alerts/{id}
PATCH /alerts/{id}/resolve
```

Validaciones

```
POST /validations (estudiante sube entregable)
GET /validations/{id}
PATCH /validations/{id}/validate (tutor aprueba o rechaza)
GET /tutors/{id}/pending-validations
```

Métricas individuales

```
GET /students/{id}/progress (motivacional - para el alumno)
GET /students/{id}/metrics (de seguimiento - para el tutor)
```

Ciclo Pedagógico Detallado

Generación del path

1. Recibe GapReport.
2. Por cada skill con status MISSING o NEEDS_WORK:
 - Crea un learning_module.
3. Ordena módulos por priority (HIGH > MEDIUM > LOW) y por dependencias.

4. Por cada modulo:
 - Genera 3 unidades base: Pasion, Play, Practica.
 - Cada unidad se llena con contenido via Gemini.
5. Persiste todo.
6. Asigna tutor (algoritmo a definir).
7. Devuelve learning_path_id al estudiante.

Ejecucion de una unidad

1. Estudiante hace GET /learning-paths/{id}/next-unit.
2. Sistema entrega la siguiente unidad pendiente.
3. Estudiante consume el contenido (texto, video, sandbox).
4. Para unidad de PRACTICA:
 - Estudiante intenta los ejercicios.
 - Cada intento dispara POST /attempts.
 - Gemini evalua el intento y devuelve ai_feedback.
 - Se actualiza skill_mastery.
5. Cuando mastery ≥ 0.80 :
 - El modulo pasa a status MASTERED.
 - Se emite evento mastery_achieved.
 - Se desbloquea el siguiente modulo.
6. Si mastery < 0.80 despues de N intentos:
 - Se cicla a una explicacion alternativa (no la misma).
 - Se refuerza con mensaje de Paciencia/Perseverancia.

Deteccion de alertas

Reglas iniciales (a refinar):

```
LOW_PERSEVERANCE
-> 5 intentos fallidos seguidos en el mismo ejercicio
-> nivel 1

ABANDON_RISK
-> 72h sin actividad
-> nivel 2
-> 7 dias sin actividad
-> nivel 3 (escalar a admin)

LOST_PASSION
-> tiempo promedio de sesion cae mas de 50% en 5 dias
-> nivel 1

ANOMALY_PATIENCE
-> lenguaje agresivo detectado por NLP en chat con el coach
-> nivel 2
```

Cada alerta queda persistida y se notifica al tutor asignado.

Integracion Con IA - Google Gemini

Por que Gemini

- Gratis con cuenta Google (sin tarjeta).
- 1M tokens/dia (suficiente para TP universitario).
- 15 req/min en plan free.
- SDK Python oficial: `google-generativeai`.
- Modelo: `gemini-1.5-flash` (rapido y barato).

Casos de uso

1. Generar contenido de unidades (Pasion, Play, Practica) contextualizado al perfil del estudiante.
2. Evaluar respuestas de ejercicios.
3. Generar feedback formativo en cada intento.
4. Reformular explicaciones alternativas cuando el estudiante no alcanza el dominio.
5. Detectar tono emocional en mensajes (Paciencia).

Configuracion

Variable de entorno:

```
GEMINI_API_KEY=...
GEMINI_MODEL=gemini-1.5-flash
```

Cliente envuelto en `app/shared/llm.py` para poder cambiar el proveedor sin tocar la logica de negocio.

Fallback

Si Gemini falla o esta sin cuota:

- Logear el error.
- Usar contenido estatico precargado por skill (un fallback minimo).
- Notificar al admin.

Metricas Individuales

Para el estudiante (motivacional)

Metrica	Como se muestra
% completado del path	"Vas por el 40%"
Habilidades dominadas	"Dominaste Git, te faltan SQL y HTTP APIs"
Streak de dias activos	"7 dias seguidos aprendiendo"
Tiempo invertido	"12 horas de practica"
Proximo hito	"Te faltan 3 ejercicios para dominar SQL"

Importante: no se muestra el `readiness_score` numerico crudo.

Para el tutor (seguimiento)

Metrica	Para que
Frecuencia de sesiones	Detectar caida de actividad
Duracion promedio de sesion	Detectar sesiones cada vez mas cortas
Dias de inactividad	Disparar alerta de abandono
Tasa de exito en ejercicios	Detectar frustracion
Reintentos por unidad	Detectar bloqueo
Ratio teoria/practica	Detectar pasividad
Habilidades estancadas	Saber donde reforzar manualmente

Eventos Emitidos Al Equipo 1

El Equipo 1 calcula metricas globales leyendo eventos. Lista inicial:

```
session_started
session_ended
exercise_attempted
exercise_completed
unit_completed
module_completed
path_completed
mastery_achieved
alert_raised
tutor_intervened
validation_submitted
validation_approved
```

Cada evento se persiste en la tabla `events`. El Equipo 1 puede leerla directamente o se expone un endpoint `GET /events` (a decidir).

Decisiones Pendientes

1. Tipo de respuesta del estudiante: texto libre, multiple opcion o codigo.
2. Orden de fases 5P: fijo o decidido por IA.
3. Algoritmo de asignacion de tutor: manual, round-robin, por especialidad.
4. Cantidad de intentos antes de cambiar a explicacion alternativa.
5. Umbral exacto para cada nivel de alerta (los listados son una propuesta inicial).
6. Como se entrega el contenido inicial (texto plano via API, markdown, HTML).

Aplicacion De La Matriz De Accesos

Ver [access-matrix.md](#) . Resumen aplicado al modulo:

- Estudiante: solo lo propio, sin ver score crudo, sin ver alertas.
- Tutor: solo sus asignados, ve alertas y valida entregables.
- Empresa: para MVP no ve nada del avance. Solo carga ofertas.
- Admin: todo.

Estado MVP

Pieza	Estado
Modelo de datos	Disenado, sin migrar
Endpoints	Disenados, sin implementar
Integracion Gemini	Decidida, sin codear
Sin login	OK para MVP, mock con headers
Contrato con Eq1	OK (GapReport definido)
Eventos hacia Eq1	Por acordar

Diseño de recursos

Cómo el sistema enriquece cada unit del plan con material externo (videos, guías, sandboxes) sin alucinar URLs.

Qué resuelve

El `content` y los `exercises` generados por el LLM cubren lo que el alumno lee y practica formalmente. Falta el material complementario que pide el modelo 5P: videos cortos, sandboxes interactivos, documentación de referencia. Estos son recursos externos — el sistema los referencia, no los aloja.

Cómo funciona

Tabla `resources` con clave funcional `(skill_name, phase)`. Cuando se genera un plan, por cada (skill, fase) se ejecuta `resources_service.get_or_suggest`:

1. Si hay rows activas en BD → reutilizar.
2. Si no hay → pedir al LLM activo, persistir, devolver.

Resultado: cada combinación skill/fase paga UNA llamada al LLM en toda su vida. Dos planes con la misma skill comparten los mismos recursos.

Los recursos se embeben en el `plan_json` al guardar el plan, igual que los exercises. Quedan congelados con el plan.

Anti-alucinación de URLs

Los LLMs inventan URLs de YouTube. Para evitarlo:

- Al LLM le pedimos **título + canal/fuente + términos de búsqueda**, no URL exacta.
- Para docs oficiales estables hay una whitelist (`_KNOWN_DOC_DOMAINS` en `suggester.py`: MDN, react.dev, git-scm.com, freecodecamp.org, etc.). Si el LLM devuelve una `canonical_url` que matchea esos dominios, la usamos.
- Para todo lo demás (especialmente videos) armamos URL de búsqueda:
`https://www.youtube.com/results?search_query=...`. Esos links nunca mueren — el alumno hace un click extra a la página de resultados.

Cantidades por fase

Fase	Cuántos	Tipos
<code>pasion</code>	2	video motivacional + lectura introductoria
<code>play</code>	2	sandbox interactivo + tutorial corto
<code>practica</code>	3	guía oficial + cheatsheet + plataforma de ejercicios

Tabla

```
CREATE TABLE resources (
  id SERIAL PRIMARY KEY,
  skill_name VARCHAR(255) NOT NULL,
  phase VARCHAR(20) NOT NULL,
  type VARCHAR(20) NOT NULL,          -- video | guide | sandbox | reading
  title VARCHAR(500) NOT NULL,
  url TEXT NOT NULL,
  description TEXT,
  duration_minutes INTEGER,
  source VARCHAR(100),
  language VARCHAR(5) DEFAULT 'es',
  level VARCHAR(20),                  -- beginner | intermediate | advanced
  generated_by VARCHAR(20) DEFAULT 'manual',
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

`is_active` permite desactivar un recurso sin borrarlo (preserva auditoría).

Manejo de fallas

Falla	Comportamiento
LLM no configurado	<code>suggester</code> devuelve <code>[]</code> , las units quedan sin <code>resources</code>
LLM falla (cuota, timeout)	<code>[]</code> , las units quedan sin <code>resources</code>
JSON malformado del LLM	Items inválidos se skippean

El POST nunca falla por culpa de los recursos. Si no hay, el alumno ve el plan sin material extra.

Cómo regenerar recursos de una skill

```
DELETE FROM resources WHERE skill_name = 'JavaScript';
```

El próximo plan con esa skill vuelve a llamar al LLM.

Lo que no está

- Endpoints CRUD para administrar el catálogo. Para curar manualmente hoy se usa SQL directo. Cuando haga falta, sumar `app/modules/resources/router.py` (el service y repository ya están listos).
- Constraint UNIQUE `(skill_name, phase, url)` . En alta concurrencia podría duplicar. No bloquea ni rompe; suma rows.
- TTL del cache. Los recursos quedan vigentes para siempre hasta borrado manual.

Matriz de accesos

Define que rol puede hacer que en el sistema. Sirve para:

- Diseñar endpoints respetando permisos desde el día 1.
- Que el frontend sepa que pantallas mostrar a cada rol.
- Implementar auth cuando llegue el momento sin rediseñar.

Roles

Rol	Quien
student	Estudiante que cursa la capacitacion
tutor	Acompañante humano de un grupo de estudiantes
company_admin	Referente de una empresa con ofertas laborales
admin	Operacion interna de la plataforma

Convenciones

- ALL -> acceso total
- OWN -> solo lo propio (su perfil, sus datos)
- READ -> solo lectura
- WRITE_OWN -> escritura solo sobre lo propio
- WRITE_ASSIGNED -> escritura solo sobre estudiantes asignados (tutor)
- NONE -> sin acceso

Nota: Para el MVP, la empresa NO ve informacion del avance de estudiantes. Solo carga ofertas y nada mas. La visibilidad de la empresa sobre aplicantes queda para post-MVP.

Matriz Por Entidad

Cuenta propia

Accion	Student	Tutor	Empresa	Admin
Ver mi perfil	OWN	OWN	OWN	ALL
Editar mi perfil	OWN	OWN	OWN	ALL

Empresas

Accion	Student	Tutor	Empresa	Admin
Listar empresas	READ	READ	OWN	ALL
Ver empresa	READ	READ	OWN	ALL
Crear empresa	NONE	NONE	ALL	ALL
Editar empresa	NONE	NONE	OWN	ALL

Ofertas y posiciones

Accion	Student	Tutor	Empresa	Admin
Listar ofertas	READ	READ	OWN	ALL
Crear oferta	NONE	NONE	ALL	ALL
Editar oferta	NONE	NONE	OWN	ALL

Estudiantes

Accion	Student	Tutor	Empresa	Admin
Listar estudiantes	NONE	READ (asignados)	NONE	ALL
Ver perfil estudiante	OWN	READ (asignados)	NONE	ALL
Crear estudiante	NONE	NONE	NONE	ALL (via form Google)

Gap Report

Accion	Student	Tutor	Empresa	Admin
Ver mi gap	OWN	READ (asignados)	NONE	ALL
Generar gap	NONE	NONE	NONE	ALL (auto al elegir oferta)

Learning Path

Accion	Student	Tutor	Empresa	Admin
Ver mi path	OWN	READ (asignados)	NONE	ALL
Avanzar en path (ejercicios)	OWN	NONE	NONE	NONE
Modificar path	NONE	WRITE_ASSIGNED	NONE	ALL

Sesiones y attempts

Accion	Student	Tutor	Empresa	Admin
Crear sesion / intentar ejercicio	OWN	NONE	NONE	NONE
Ver historial de intentos	OWN	READ (asignados)	NONE	ALL

Tutores

Accion	Student	Tutor	Empresa	Admin
Listar tutores	NONE	READ	NONE	ALL
Asignar tutor a estudiante	NONE	NONE	NONE	ALL

Alertas (riesgo abandono, frustracion, etc.)

Accion	Student	Tutor	Empresa	Admin
Ver alertas	NONE	READ (asignados)	NONE	ALL
Resolver alerta	NONE	WRITE_ASSIGNED	NONE	ALL

Nota: el estudiante NO ve sus propias alertas. Si las viera el sistema perderia su rol motivacional.

Validaciones criticas

Accion	Student	Tutor	Empresa	Admin
Subir entregable	OWN	NONE	NONE	NONE
Validar entregable	NONE	WRITE_ASSIGNED	NONE	ALL
Ver feedback	OWN	READ (los que valido)	NONE	ALL

Metricas

Accion	Student	Tutor	Empresa	Admin
Ver mi progreso	OWN	NONE	NONE	NONE
Ver progreso de mis asignados	NONE	READ	NONE	ALL
Ver metricas globales del sistema	NONE	NONE	NONE	ALL

Reglas Que Se Desprenden

1. El estudiante solo se ve a si mismo. No conoce a otros estudiantes ni tutores.

2. El tutor solo ve a sus asignados. No puede meterse en estudiantes ajenos.
3. La empresa para MVP NO ve avance de estudiantes. Solo carga sus ofertas.
4. El admin ve y puede todo. Es el rol operativo.
5. Solo el estudiante puede avanzar en su propio path.
6. El tutor puede modificar el path del asignado (agregar refuerzo, saltar unidad).
7. Las alertas son internas. El estudiante no las ve.

Casos Delicados

A - Visibilidad de la empresa sobre el aprendizaje

Para MVP, la empresa NO ve avance de estudiantes. Solo carga sus ofertas.

Para post-MVP, la empresa podra ver un RESUMEN del path del aplicante (% completado, habilidades dominadas), no el detalle de intentos. Para cuidar la dignidad del aprendiz y no exponer sus dificultades al futuro empleador.

B - Score numerico al estudiante

El estudiante NO ve el `readiness_score` numerico crudo. Ve el progreso en terminos motivacionales:

```
"Dominaste 3 de 7 habilidades"  
"Te faltan 2 ejercicios para dominar SQL"
```

El score numerico queda para tutor, empresa y admin.

C - Cambio de tutor por parte del estudiante

Para MVP el estudiante no puede pedir cambio de tutor. Funcionalidad post-MVP.

D - Multiples company_admin por empresa

Para MVP, todos los company_admin de una misma empresa tienen los mismos permisos. A futuro pueden haber niveles (supervisor, recruiter, etc.).

Implementacion EnCodigo

En MVP sin login, se simula el usuario con headers:

```
X-User-Id: 1  
X-User-Role: student
```

Cada endpoint se marca con el rol o roles requeridos. Sugerencia inicial:

```
@router.get("/learning-paths/{id}")  
# allowed: student:own | tutor:assigned | company_admin:applicants | admin:all
```



```
def get_learning_path(id: int):  
    ...
```

Cuando se suma `auth/`, se reemplaza el header mock por un decoder JWT que llena `current_user` con role y permisos. Los chequeos quedan iguales.

Resumen

- 4 roles definidos.
- Visibilidad estricta por defecto.
- Sin login en MVP, pero con la matriz aplicable apenas se suma `auth/`.
- El estudiante ve solo lo propio, en terminos motivacionales.
- Las alertas son internas para tutor y admin, nunca para el estudiante.

Guía para el frontend

Cómo armar la experiencia del alumno (mostrar plan, resolver ejercicios, continuar donde quedó) usando los endpoints actuales del backend, sin tocar código del lado del servidor.

Lo que el backend te da hoy

Tres endpoints es todo lo que necesitás:

#	Endpoint	Para qué
1	<code>GET /api/students/{email}/learning-paths</code>	Traer el/los planes del alumno
2	<code>GET /api/students/{email}/attempts</code>	Traer todo lo que ya respondió
3	<code>POST /api/attempts</code>	Registrar una nueva respuesta

No hay endpoint "next-unit". El frontend reconstruye dónde quedó el alumno cruzando los dos GETs.

Estructura del plan

```
LearningPath
├─ id, status, generator_used
├─ modules[]
│   └─ skill_name, priority
│       └─ units[]
│           └─ exactamente 3 por módulo
│               └─ [0] phase: "pasion"      ← solo texto, exercises: []
│               └─ [1] phase: "play"      ← solo texto, exercises: []
│               └─ [2] phase: "practica"   ← 5 ejercicios multiple_choice
│                   └─ exercises[]
│                       └─ prompt (con 4 opciones A/B/C/D embebidas)
│                       └─ type: "multiple_choice"
│                       └─ expected_answer: "B" ← una letra
│                       └─ difficulty: 1-5
```

Solo la unit de Práctica tiene ejercicios para responder. Pasión y Play son contenido de lectura.

Cada unit también trae `resources[]` con links a videos/guías externas (YouTube, MDN, FreeCodeCamp).

Identificación del ejercicio

Cada attempt se identifica por 4 índices compuestos. Cuando el alumno responde el ejercicio 3 del módulo 1 unit 2, el body del POST es:

```
{
  "student_email": "sofia@example.com",
  "learning_path_id": 17,
  "module_index": 1,
  "unit_index": 2,
  "exercise_index": 3,
  "answer": "C"
}
```

El backend busca el `expected_answer` original, compara, calcula `is_correct`, genera feedback empático con IA y persiste.

Flujo "continuar donde dejé"

Al entrar el alumno

1. `fetch GET /api/students/{email}/learning-paths` → lista de planes. Tomar el más reciente (o el `status === "ACTIVE"`).
2. `fetch GET /api/students/{email}/attempts` → array de attempts.
3. Mapear los attempts del plan actual a un Set de claves `${module_index}-${unit_index}-${exercise_index}` (filtrando solo `is_correct === true`).
4. Recorrer `modules → units → exercises` del plan. El primer ejercicio cuya clave no esté en el Set es "el siguiente".

Función de cálculo

```
function findNextExercise(plan, attempts) {
  const completed = new Set(
    attempts
      .filter(a => a.learning_path_id === plan.id && a.is_correct)
      .map(a => `${a.module_index}-${a.unit_index}-${a.exercise_index}`)
  );

  for (let m = 0; m < plan.modules.length; m++) {
    const units = plan.modules[m].units;
    for (let u = 0; u < units.length; u++) {
      if (units[u].phase !== "practica") continue;
      for (let e = 0; e < units[u].exercises.length; e++) {
        const key = `${m}-${u}-${e}`;
        if (!completed.has(key)) {
          return {
            moduleIndex: m,
            unitIndex: u,
```

```

        exerciseIndex: e,
        exercise: units[u].exercises[e],
        skillName: plan.modules[m].skill_name,
    };
    }
}
}
}

return null; // plan terminado
}

```

Al responder el ejercicio

```

async function submitAnswer(plan, next, answer, email) {
  const res = await fetch("/api/attempts", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      student_email: email,
      learning_path_id: plan.id,
      module_index: next.moduleIndex,
      unit_index: next.unitIndex,
      exercise_index: next.exerciseIndex,
      answer, // "A", "B", "C" o "D"
    }),
  });
  return res.json();
  // result.is_correct → true/false
  // result.ai_feedback → mensaje empático si falló
  // result.skill_mastery → 0.0 a 1.0
}

```

Después del POST, **refrescar los attempts** y volver a calcular el `next`. El nuevo `next` ya no incluye el ejercicio recién acertado.

UI sugerida — 4 pantallas

Pantalla 1 — "Mis planes"

Listar como cards:

- Empresa + Rol
- `readiness_score_initial` (barra de progreso)
- `generator_used` (badge `groq` / `gemini` / `mock`)
- `created_at`

Click en una card → Pantalla 2.

Pantalla 2 — "Plan en curso" (overview)

Combinar plan + attempts:

- Progreso global: X de Y ejercicios completados.
- Listar módulos con su `skill_name` + `priority`.
- Indicar cuántas units terminó el alumno por módulo.
- Botón grande "Continuar" → te lleva al next exercise (Pantalla 4).

Pantalla 3 — "Leer contenido" (fases Pasión y Play)

Para units con `phase === "pasion"` o `"play"`:

- Mostrar `title` como heading grande.
- Mostrar `content` como texto largo formateado.
- Mostrar `resources[]` como cards con:
- icono según `type` (`video` / `guide` / `sandbox` / `reading`)
- `title` + `source`
- botón "Abrir" → `window.open(url, "_blank")`
- Botón "Siguiente" → marca como leído (client-side, no se persiste).

Pantalla 4 — "Resolver ejercicio" (fase Práctica)

Mostrar:

- `exercise.prompt` formateado (parsear las opciones A/B/C/D del texto).
- 4 botones radio para elegir respuesta.
- Botón "Responder" → `POST /api/attempts`.
- Después de responder:
- Mostrar `is_correct` (✓ verde o ✗ rojo).
- Mostrar `ai_feedback` en una card destacada.
- Mostrar `skill_mastery` como porcentaje + barra.
- Botón "Siguiente ejercicio" → calcular nuevo `next` y mostrarlo.

Parser del prompt del ejercicio

El `prompt` del ejercicio viene con formato:

```
¿Cuál es la sintaxis correcta para declarar una constante en JavaScript?  
A) var x = 5;  
B) let x = 5;  
C) const x = 5;  
D) constant x = 5;
```

Para mostrarlo bonito, dividir:

```
function parsePrompt(prompt) {  
  const lines = prompt.split("\n");  
  const question = lines[0];
```

```

const options = lines.slice(1).map(line => {
  const match = line.match(/^[A-D])\s*(.+)/);
  return match ? { letter: match[1], text: match[2] } : null;
}).filter(Boolean);
return { question, options };
}

```

Y renderizar:

```

<h2>{question}</h2>
{options.map(opt => (
  <label key={opt.letter}>
    <input type="radio" name="answer" value={opt.letter} />
    <strong>{opt.letter}</strong> {opt.text}
  </label>
))}

```

Componente conceptual

```

function PlanEnCurso({ email }) {
  const [path, setPath] = useState(null);
  const [attempts, setAttempts] = useState([]);
  const [next, setNext] = useState(null);

  useEffect(() => {
    Promise.all([
      fetch(`/api/students/${email}/learning-paths`).then(r => r.json()),
      fetch(`/api/students/${email}/attempts`).then(r => r.json()),
    ]).then(([paths, atts]) => {
      const active = paths[paths.length - 1];
      setPath(active);
      setAttempts(atts);
      setNext(findNextExercise(active, atts));
    });
  }, [email]);

  async function handleAnswer(answer) {
    await fetch("/api/attempts", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        student_email: email,
        learning_path_id: path.id,
        module_index: next.moduleIndex,
        unit_index: next.unitIndex,
        exercise_index: next.exerciseIndex,
        answer,
      })
    });
    const newAtts = await fetch(`/api/students/${email}/attempts`).then(r => r.json());
    setAttempts(newAtts);
  }
}

```

```

    setNext(findNextExercise(path, newAtts));
  }

  if (!path) return <Loading />;
  if (!next) return <PlanCompletado />;

  return (
    <div>
      <ProgressBar value={calculateProgress(path, attempts)} />
      <Ejercicio data={next.exercise} onSubmit={handleAnswer} />
    </div>
  );
}

```

Persistencia automática del "continuar donde paró"

No hay que hacer nada especial. El estado vive en la base de datos:

- Cada `POST /api/attempts` se persiste inmediatamente.
- Cuando el alumno vuelve a entrar (en otra sesión, otro device, lo que sea), basta con re-llamar a los GETs.
- El cálculo de "next exercise" siempre devuelve lo correcto porque se basa en los datos guardados.

Si el alumno responde, cierra el browser y vuelve al día siguiente → al cargar la página, el flujo de los 3 pasos del principio lo deja exactamente donde quedó.

Limitaciones a conocer

1. **No hay "saltar unit".** Solo Práctica registra avance. Pasión y Play son "lectura libre", el cliente decide si las muestra o no.
2. **No hay "completado" formal a nivel de unit/módulo.** Una unit se considera "hecha" cuando todos sus ejercicios están acertados. Si querés un porcentaje por unit, hay que calcularlo del lado del cliente: `unitProgress = ejerciciosAcertadosEnUnit / ejerciciosTotalesEnUnit`
3. **Si el alumno falla un ejercicio, sigue contando como "pendiente".** Puede re-intentarlo. `findNextExercise` lo trae de nuevo hasta que acierte.
4. **Mastery acumulado por skill:** cada attempt response trae `skill_mastery` (entre 0.0 y 1.0). Si querés mostrar "Sofía está 60% dominando SQL", usás ese valor del último attempt en esa skill.
5. **Mastery threshold (80%) no bloquea nada.** Hoy `mastery_threshold_reached: true` es solo informativo. El alumno puede seguir avanzando aunque no domine. Si querés "no dejarlo avanzar al siguiente módulo hasta mastery 80%", eso es lógica del frontend.

Qué se puede implementar HOY desde el frontend

-  Cargar planes del alumno

- ✓ Calcular qué ejercicio sigue
- ✓ Mostrar el ejercicio con sus opciones
- ✓ Persistir respuesta con feedback de IA
- ✓ Mostrar progreso (% completado, mastery por skill)
- ✓ Persistencia entre sesiones (es automática)
- ✓ Mostrar contenido de fases Pasión/Play
- ✓ Mostrar recursos externos (links a YouTube, MDN, etc.)

Lo que **no** se puede hoy y requiere trabajo de backend (ver [backlog.md](#)):

- ✗ Tracking de tiempo en cada unit (falta módulo [sessions](#) — Épica 1)
 - ✗ Endpoint [/next-unit](#) empaquetado (hoy lo calcula el cliente — Épica 4)
 - ✗ Streak de días activos (necesita [sessions](#))
 - ✗ Bloqueo de avance hasta mastery 80% (lógica server-side — Épica 4)
-

Referencias

- Curls listos para probar contra producción: [api-curls.md](#).
- Diseño del flujo pedagógico 5P y qué cubre el sistema: [5p-coverage.md](#).
- Backlog con épicas pendientes que liberarían funcionalidad nueva en el backend: [backlog.md](#).

[Capacity AR](#)[api-curls.md](#)

Curls de prueba de la API

Guía de prueba de la API deployada para que los compañeros del equipo puedan probar los endpoints sin tener que levantar nada local.

Links principales

- **API base:** <https://capacity-ar-ap.onrender.com>
- **Swagger UI (probar desde el navegador):** <https://capacity-ar-ap.onrender.com/docs>
- **Health check:** <https://capacity-ar-ap.onrender.com/api/health>

Antes de empezar

El plan free de Render **duerme el servidor tras 15 minutos sin tráfico**. El primer request después de un rato tarda entre 30 y 50 segundos en despertar el contenedor. Los siguientes vuelan.

Para "calentar" el server antes de una prueba o demo:

```
curl https://capacity-ar-ap.onrender.com/api/health
```

Cuando devuelva `{"status":"ok","environment":"production"}` ya está listo.

Endpoints disponibles

Users

Método	Ruta	Descripción
GET	/api/users	Listar usuarios (con paginación)
POST	/api/users	Crear usuario
GET	/api/users/{id}	Obtener un usuario por id
PATCH	/api/users/{id}	Actualizar parcialmente
DELETE	/api/users/{id}	Borrar usuario

Roles válidos: [student](#), [tutor](#), [company_admin](#), [admin](#).

Gap analyses (carga de documentos → summary)

Método	Ruta	Descripción
POST	/api/gap-analyses	Subir 2 documentos (estudiante + oferta), Groq extrae GapReport, guarda en BD. NO dispara plan.
GET	/api/gap-analyses/{id}	Ver un gap por id
GET	/api/students/{email}/gap-analyses	Listar gaps de un estudiante
POST	/api/students/{email}/generate-learning-path	A partir del último gap pendiente del estudiante, dispara la generación del learning_path

Learning paths

Método	Ruta	Descripción
POST	/api/learning-paths	Crear plan a partir de un GapReport JSON (sin documentos)
GET	/api/learning-paths	Listar planes
GET	/api/learning-paths/{id}	Obtener un plan por id
GET	/api/students/{email}/learning-paths	Listar planes de un estudiante

Attempts (ejercicios resueltos por el alumno)

Método	Ruta	Descripción
POST	/api/attempts	Registrar intento, evalúa y devuelve feedback con IA
GET	/api/attempts/{id}	Ver un attempt
GET	/api/students/{email}/attempts	Historial de un estudiante

USERS — curls

Crear

```
curl -X POST https://capacity-ar-ap.onrender.com/api/users \
  -H "Content-Type: application/json" \
  -d
'{"first_name":"Ana","last_name":"Perez","email":"ana@example.com","role":"student"}
'
```

Roles válidos: `student`, `tutor`, `company_admin`, `admin`. Reintento con el mismo email devuelve `409`.

Listar / ver / actualizar / borrar

```
curl https://capacity-ar-ap.onrender.com/api/users
curl "https://capacity-ar-ap.onrender.com/api/users?offset=0&limit=10"
curl https://capacity-ar-ap.onrender.com/api/users/1
curl -X PATCH https://capacity-ar-ap.onrender.com/api/users/1 \
  -H "Content-Type: application/json" -d '{"last_name":"Gonzalez"}'
curl -X DELETE https://capacity-ar-ap.onrender.com/api/users/2
```

`GET /api/users/9999` → `404`. `DELETE` exitoso → `204` sin body.

GAP ANALYSES — curls

Flujo en dos pasos:

1. **Subir documentos** del estudiante y de la oferta → Groq extrae un GapReport y lo guarda. Rápido (3-5s). NO dispara plan.
2. **Disparar el plan más tarde** identificando al estudiante por email. El sistema busca el último gap pendiente y llama a `learning_paths`. Lento (15-25s), pero aislado del primer paso.

Aceptamos `PDF`, `DOCX` y `TXT`. Máximo 5MB por archivo, mínimo 50 caracteres tras extraer texto.

Paso 1 — Subir documentos y generar summary

```
curl -X POST https://capacity-ar-ap.onrender.com/api/gap-analyses \
  -F "student_email=sofia.maldonado@example.com" \
  -F "company_email=data-talent@globant.com" \
  -F "student_doc=@/ruta/al/cv_sofia.pdf" \
  -F "position_doc=@/ruta/a/oferta_globant.pdf"
```

Respuesta `201` con `id`, `summary`, `readiness_score`, `gap_report.skills` calculadas, `learning_path_id: null`.

Tip rápido: podés probar con dos `.txt`. Acordate del `@` antes de la ruta (es la convención de curl para subir archivos).

Paso 2 — Disparar el plan a partir del email

Toma el último gap pendiente (`learning_path_id IS NULL`) del estudiante:

```
curl -X POST https://capacity-ar-ap.onrender.com/api/students/sofia.maldonado@example.com/generate-learning-path
```

Respuesta `201` con `{ gap_analysis, learning_path }`. El `gap_analysis` ahora tiene `learning_path_id` apuntando al plan recién creado.

Consultar / listar

```
# Un gap por id
curl https://capacity-ar-ap.onrender.com/api/gap-analyses/1

# Todos los gaps de un estudiante (devuelve array, ordenado del más reciente al más viejo)
curl https://capacity-ar-ap.onrender.com/api/students/sofia.maldonado@example.com/gap-analyses
```

Casos de error

- Archivo con extensión distinta a `.pdf/.docx/.txt` → `422`.
- Archivo > 5MB → `413`.
- Texto extraído < 50 caracteres (PDF escaneado sin OCR, archivo casi vacío) → `422`.
- Email inválido → `422`.
- `generate-learning-path` sin gap pendiente para ese email → `404`.
- Groq devuelve JSON inválido o falla → `502` con detalle.

Cómo funciona con varios summaries por email

El alumno puede subir documentos varias veces — cada call crea una fila nueva. Cuando llama a `generate-learning-path`, el sistema toma el **último gap con `learning_path_id IS NULL`** (LIFO). Si quiere generar planes para los anteriores también, vuelve a llamar al endpoint hasta que devuelva `404`.

Verificación en Neon

```
-- Resumen de gaps por estudiante
SELECT id, student_email, company_email, readiness_score,
       learning_path_id, created_at
FROM gap_analyses
ORDER BY id DESC LIMIT 10;

-- Ver el GapReport completo extraído por Groq
SELECT id, gap_report_json FROM gap_analyses WHERE id = 1;

-- Conectar gap con el plan generado
SELECT ga.id AS gap_id, ga.student_email, ga.summary,
       lp.id AS plan_id, lp.target_role_title
FROM gap_analyses ga
LEFT JOIN learning_paths lp ON lp.id = ga.learning_path_id
ORDER BY ga.id DESC;
```

LEARNING PATHS — curls

Crear plan desde un GapReport

```
curl -X POST https://capacity-ar-ap.onrender.com/api/learning-paths \
-H "Content-Type: application/json" \
-d '{
  "student": {
    "name": "Ana Perez",
    "email": "ana@example.com",
    "skills": [{ "name": "Git", "level": 1 }, { "name": "SQL", "level": 2 }],
    "interests": ["backend", "logistica"]
  },
  "company": { "name": "Empresa Logistica SA" },
  "target_role": {
    "title": "Backend Developer Junior",
    "required_skills": [
      { "name": "Git", "level": 3, "priority": "HIGH" },
      { "name": "SQL", "level": 3, "priority": "HIGH" },
      { "name": "HTTP APIs", "level": 2, "priority": "MEDIUM" }
    ]
  },
  "summary": "Ana necesita reforzar Git, SQL y HTTP APIs.",
  "readiness_score": 45,
  "skills": [
    { "name": "Git", "current_level": 1, "required_level": 3, "gap_level": 2,
      "priority": "HIGH", "status": "MISSING" },
    { "name": "SQL", "current_level": 2, "required_level": 3, "gap_level": 1,
      "priority": "HIGH", "status": "NEEDS_WORK" },
    { "name": "HTTP APIs", "current_level": 0, "required_level": 2, "gap_level": 2,
      "priority": "MEDIUM", "status": "MISSING" }
  ]
}'
```

Respuesta `201` con `id`, `status: "ACTIVE"`, `generator_used`, módulos por skill MISSING/NEEDS_WORK y cada módulo con units `pasion` / `play` / `practica`.

Listar / ver / por estudiante

```
curl https://capacity-ar-ap.onrender.com/api/learning-paths
curl https://capacity-ar-ap.onrender.com/api/learning-paths/1
curl https://capacity-ar-ap.onrender.com/api/students/ana@example.com/learning-paths
```

Casos de error

- Todas las skills en `READY` → `409 Conflict` (no genera plan vacío).
- `GET /api/learning-paths/9999` → `404`.
- Payload con campos faltantes → `422` con detalle del campo.

ATTEMPTS — curls

Identificación del ejercicio: 4 campos compuestos (`learning_path_id` + `module_index` + `unit_index` + `exercise_index`). El servidor abre el plan, navega esa posición y compara con el `expected_answer` original.

Responder un ejercicio

```
curl -X POST https://capacity-ar-ap.onrender.com/api/attempts \
-H "Content-Type: application/json" \
-d '{
  "student_email": "carlos.gamer@example.com",
  "learning_path_id": 7,
  "module_index": 0,
  "unit_index": 2,
  "exercise_index": 0,
  "answer": "B"
}'
```

Respuesta:

- `is_correct` + `score` (1.0 o 0.0).
- `ai_feedback`: mensaje determinístico si acertó, generado por IA en español rioplatense si falló (Paciencia — no revela la respuesta correcta, ofrece pista, invita a reintentar).
- `skill_mastery`: aciertos / total intentos en esa skill.
- `mastery_threshold_reached: true` cuando `skill_mastery >= 0.80`.

Consultar

```
curl https://capacity-ar-ap.onrender.com/api/attempts/1
curl https://capacity-ar-
ap.onrender.com/api/students/carlos.gamer@example.com/attempts
```

Errores esperados

- `learning_path_id` que no existe → `404`.
- Email del attempt distinto al del plan → `403 Forbidden`.

Mastery por skill (Neon)

```
SELECT skill_name,
       COUNT(*) AS attempts,
       SUM(CASE WHEN is_correct THEN 1 ELSE 0 END) AS aciertos,
       ROUND(SUM(CASE WHEN is_correct THEN 1.0 ELSE 0.0 END) / COUNT(*), 2) AS
mastery
FROM attempts
WHERE student_email = 'carlos.gamer@example.com'
GROUP BY skill_name;
```

RESOURCES — cómo verlos

Sin endpoints CRUD propios. Los recursos viajan dentro de cada `unit.resources[]` del plan generado.

```
-- Catálogo cacheado
SELECT skill_name, phase, type, title, source, url
FROM resources
WHERE is_active = TRUE
ORDER BY skill_name, phase, id;

-- Regenerar recursos de una skill
DELETE FROM resources WHERE skill_name = 'JavaScript';
```

Detalles en [resources-design.md](#).

Ver cómo la IA personaliza la respuesta

En cada respuesta del `POST /learning-paths` mirar:

- `generator_used` → "groq" o "gemini" según el proveedor activo. Si dice "mock" el LLM falló y cayó al fallback.
- `modules[0].units[0].content` → debe mencionar el nombre, los intereses y la empresa objetivo del estudiante.
- `modules[0].units[2].exercises` → 5 multiple_choice reales con A/B/C/D.
- `modules[0].units[*].resources` → 2-3 recursos externos por unit (videos, guías, sandboxes).

Para mostrar variabilidad: repetir el mismo body dos veces. Como `temperature=0.7`, el LLM produce contenidos distintos cada vez. Es la prueba clara de que no hay cache ni contenido hardcodedo.

Body de ejemplo (Frontend Junior con interés en videojuegos)

```
curl -X POST https://capacity-ar-ap.onrender.com/api/learning-paths \
-H "Content-Type: application/json" \
-d '{
  "student": {
    "name": "Carlos Mendez",
    "email": "carlos.gamer@example.com",
    "skills": [{ "name": "HTML", "level": 3 }],
    "interests": ["videojuegos", "esports"]
  },
  "company": { "name": "GameStudio Argentina" },
  "target_role": {
    "title": "Frontend Developer Junior",
    "required_skills": [{ "name": "JavaScript", "level": 3, "priority": "HIGH" }]
  },
  "summary": "Carlos necesita JavaScript.",
  "readiness_score": 30,
  "skills": [{ "name": "JavaScript", "current_level": 1, "required_level": 3,
```

```
"gap_level": 2, "priority": "HIGH", "status": "MISSING"}]
```

Para probar otro perfil, cambiar `student.interests`, `company.name`, `target_role.title` y las `skills` requeridas. El LLM va a adaptar las analogías y los ejercicios al nuevo contexto.

Tips

Ver status code + headers

```
curl -i https://capacity-ar-ap.onrender.com/api/users/9999
```

El `-i` muestra los headers incluyendo `HTTP/2 404`.

JSON con formato (requiere `jq`)

```
curl -s https://capacity-ar-ap.onrender.com/api/learning-paths | jq
```

Instalación: `sudo apt install jq` en Ubuntu/Debian, `brew install jq` en Mac.

Pegar todo desde Swagger en vez de curl

Entrar a <https://capacity-ar-ap.onrender.com/docs>, clicar el endpoint, "Try it out", editar el JSON y "Execute". Es lo más cómodo para una demo o para probar rápido sin terminal.

Contrato de datos

Contrato completo del GapReport y reglas de cálculo en [gap-engine-mvp.md](#).

Backlog

Vista unificada del trabajo de Equipo 2 (Capacitación + Tutores). Cada épica lista user stories, endpoints, criterio de aceptación y esfuerzo (real para las completadas, estimado para las pendientes).

Estado del modelo 5P y arquitectura: [5p-coverage.md](#), [ai-integration.md](#), [scope-equipos.md](#).

Resumen

	Épica	Estado	Esfuerzo
C1	MVP backend FastAPI + módulo users	✓ Completada	~4 h
C2	Módulo learning_paths con MockPlanGenerator	✓ Completada	~6 h
C3	Persistencia en PostgreSQL (Neon + Render)	✓ Completada	~5 h
C4	Generación con IA: Groq + Gemini con fallback	✓ Completada	~6 h
C5	Módulo attempts con Paciencia (4º P)	✓ Completada	~3 h
C6	Módulo resources con anti-alucinación de URLs	✓ Completada	~3 h
1	Perseverancia y métricas del alumno	⚠ Pendiente	3-4 h
2	Tutores y asignaciones	⚠ Pendiente	4-5 h
3	Sistema de alertas	⚠ Pendiente	3-4 h
4	Next-unit logic	⚠ Pendiente	2-3 h
5	Validaciones críticas	⚠ Pendiente	3 h
6	Auth (mock + JWT real)	⚠ Pendiente	9-12 h
7	Eventos hacia Equipo 1	⚠ Pendiente	2 h
8	Calidad técnica (tests, CORS, Alembic)	⚠ Pendiente	4-6 h

Orden sugerido para cerrar el MVP del modelo

1. Épica 1 — Perseverancia (cierra las 5P).
2. Épica 2 — Tutores (demuestra HITL).
3. Épica 3 — Alertas.
4. Épica 4 — Next-unit logic.

Las cuatro juntas dejan el sistema completo según los documentos del modelo. Total estimado: ~13-16 horas.

Épicas completadas

Épica C1 — MVP backend FastAPI + módulo users

Levantar el esqueleto del backend con el patrón de módulo que se replica en todo el sistema, y entregar el primer módulo de dominio (`users`) para registrar a los actores del modelo.

User stories:

- Como dev quiero un patrón consistente (router/service/repository/models/schemas) para que el código sea mantenible cuando crezca.
- Como admin quiero registrar usuarios con rol (`student` , `tutor` , `company_admin` , `admin`) para identificar a los actores del sistema.
- Como sistema quiero validar que no se repitan emails.

Endpoints entregados:

- `POST /users` , `GET /users` , `GET /users/{id}` , `PATCH /users/{id}` , `DELETE /users/{id}`
- `GET /health`

Criterio de aceptación cumplido:

- Email duplicado devuelve `409 Conflict` .
- ID inexistente devuelve `404` .
- Patrón de módulo documentado en `docs/backend-onboarding.md` .

Commits: `be3462d` (Initial FastAPI MVP backend).

Épica C2 — Módulo `learning_paths` con MockPlanGenerator

Implementar el primer módulo pedagógico que consume el GapReport del Equipo 1 y genera un plan con módulos por skill y unidades por fase 5P (Pasión / Play / Práctica). Sin IA todavía, para validar la estructura y el contrato.

User stories:

- Como sistema quiero recibir un GapReport y producir un plan con módulos ordenados por prioridad.
- Como sistema quiero rechazar GapReports donde todas las skills estén en READY (no hay nada que enseñar).
- Como dev quiero poder cambiar el generador (Mock → IA real) sin tocar el resto del sistema.

Endpoints entregados:

- `POST /learning-paths`

- `GET /learning-paths`
- `GET /learning-paths/{id}`
- `GET /students/{email}/learning-paths`

Criterio de aceptación cumplido:

- Contrato del GapReport alineado con `docs/gap-engine-mvp.md`.
- Patrón Protocol + factory en `plan_generator/` para soportar múltiples implementaciones.
- 3 fases 5P (Pasión / Play / Práctica) por skill MISSING o NEEDS_WORK.
- Cobertura del modelo 5P documentada en `docs/5p-coverage.md`.

Commits: `16873c0`, `2be5e49` (persistence layer Protocol + in-memory).

Épica C3 — Persistencia en PostgreSQL (Neon + Render)

Llevar el backend de in-memory a una base real y deployarlo en internet con auto-deploy desde main.

User stories:

- Como cliente externo quiero que la API esté disponible en una URL pública.
- Como sistema quiero que los datos sobrevivan a reinicios del contenedor.
- Como dev quiero auto-deploy desde main para no manejar deploys manuales.

Cambios entregados:

- Web Service en Render (free tier) con auto-deploy desde main.
- PostgreSQL en Neon (free tier serverless).
- `app/config.py` normaliza la URL al driver `postgresql+psycopg://`.
- `app/main.py` corre `Base.metadata.create_all` en el lifespan startup.
- Migración del repositorio de `learning_paths` a SQLAlchemy con `plan_json` como JSON serializado.
- Pin de Python 3.12 en `.python-version` (sin esto Render usa 3.14 y SQLAlchemy 2.0.36 rompe).

Criterio de aceptación cumplido:

- URL pública: `https://capacity-ar-ap.onrender.com`.
- Push a main dispara redeploy automático en 2-4 min.
- Datos persisten entre reinicios del contenedor.

Commits: `f3580ec`, `6d92bce`, `c67b725`, `700051c`.

Épica C4 — Generación con IA: Groq + Gemini con fallback

Reemplazar el MockPlanGenerator por LLMs reales, manteniendo la estructura híbrida (código determinístico para la estructura del plan, LLM solo para el contenido). Multi-proveedor para no depender de uno solo.

User stories:

- Como alumno quiero contenido educativo personalizado a mis intereses y empresa objetivo, generado en tiempo real.

- Como sistema quiero soportar dos proveedores de IA y elegirlos por env var.
- Como sistema quiero nunca romper la API por culpa del LLM: si falla, caer a Mock automáticamente.
- Como dev quiero pedir ejercicios estructurados (5 multiple_choice con A/B/C/D) que sean evaluables automáticamente.

Cambios entregados:

- `app/shared/llm.py` con `GeminiClient` y `GroqClient` (misma interfaz).
- `plan_generator/_base_llm.py` con clase base abstracta que paraleliza las llamadas con `ThreadPoolExecutor`.
- `GeminiPlanGenerator` y `GroqPlanGenerator` como sub-clases de 4 líneas.
- Factory que lee env var `PLAN_GENERATOR` y envuelve en `_PlanGeneratorWithMockFallback`.
- Prompts versionados en `prompts.py` con instrucciones específicas por fase 5P (tono rioplatense, conexión con intereses, ejercicios A/B/C/D).
- Schema de respuesta del LLM declarado explícitamente para evitar features que Gemini no soporta.

Criterio de aceptación cumplido:

- `PLAN_GENERATOR=groq` genera plan con `generator_used: "groq"` en 5-15s.
- `PLAN_GENERATOR=gemini` genera plan equivalente con `generator_used: "gemini"`.
- Si el LLM falla (cuota, timeout), el plan se devuelve con `generator_used: "mock"`, sin romper.
- 5 ejercicios multiple_choice por skill con `expected_answer` como letra única.
- Cambiar de proveedor en producción no requiere tocar código.

Commits: `7209bb5`, `c2b1e14`, `d5083a5`.

Épica C5 — Módulo attempts con Paciencia (4° P)

Cerrar el ciclo del alumno: que pueda responder ejercicios, recibir feedback empático y avanzar según mastery 80%. Activa la 4° P del modelo 5P de forma transversal.

User stories:

- Como alumno quiero responder un ejercicio del plan y saber si acerté.
- Como alumno quiero recibir un mensaje empático cuando me equivoco, sin que me revelen la respuesta.
- Como sistema quiero calcular el mastery por skill (aciertos / intentos) y marcar cuando supera el 80%.
- Como alumno quiero ver mi historial de intentos.
- Como sistema quiero rechazar intentos sobre planes que no son del alumno (403).

Endpoints entregados:

- `POST /attempts`
- `GET /attempts/{id}`
- `GET /students/{email}/attempts`

Cambios entregados:

- Tabla `attempts` con `learning_path_id` + `module_index` + `unit_index` + `exercise_index` como identificador compuesto (sin tocar el `plan_json`).

- Evaluación por string match (sirve para multiple_choice).
- `attempts/feedback.py` con generación de feedback empático vía LLM activo (Groq o Gemini).
- Fallback a mensaje estático si el LLM falla.

Criterio de aceptación cumplido:

- Respuesta correcta: `is_correct: true`, mensaje de felicitación determinístico con `skill_mastery`.
- Respuesta incorrecta: `is_correct: false`, mensaje empático generado por IA que NO revela la respuesta esperada.
- Intento sobre plan ajeno: `403 Forbidden`.
- Mastery se recalcula al consultar el historial (no se persiste, se computa).

Commits: `6a33951`.

Épica C6 — Módulo resources con anti-alucinación de URLs

Enriquecer cada unit del plan con material de estudio externo (videos, guías, sandboxes). Generado por LLM pero con guardrails para que los links no apunten a recursos inexistentes.

User stories:

- Como alumno quiero ver videos, guías y sandboxes asociados a cada unit del plan.
- Como sistema quiero cachear los recursos por (skill, fase) para no llamar al LLM cada vez.
- Como sistema quiero garantizar que los links generados no sean URLs alucinadas por el LLM.

Cambios entregados:

- Tabla `resources` con clave funcional `(skill_name, phase)`.
- `resources/suggester.py` con prompt que pide al LLM `title + source + search_terms` (no URL directa).
- Whitelist de dominios estables (`_KNOWN_DOC_DOMAINS`): MDN, git-scm.com, react.dev, freecodecamp.org, docs.python.org, etc.
- Para videos: armado de URL como `https://www.youtube.com/results?search_query=...` (búsqueda, no link directo).
- Para docs canónicas: URL aceptada solo si matchea la whitelist.
- Pattern cache-aside en `resources/service.py`: si hay rows en BD reutilizar, si no pedir al LLM y persistir.
- Embed de los recursos dentro del `plan_json` al generar el plan.

Criterio de aceptación cumplido:

- Cada unit del plan generado trae 2-3 recursos en `unit.resources[]`.
- Las URLs de videos son links de búsqueda (nunca mueren).
- Las URLs de documentación canónica apuntan a dominios reales whitelisteados.
- Segundo plan con misma skill reutiliza los mismos recursos (cero llamadas al LLM).

Commits: `5c0f901`.

Épicas pendientes

Épica 1 — Perseverancia y métricas del alumno

Cierra la 5ª P del modelo (Perseverancia) implementando la visualización del progreso y el tracking de actividad.

Módulos a crear: `sessions` + `student_metrics`.

User stories:

- Como alumno quiero ver mi progreso (% completado, hitos, streak de días) para mantenerme motivado, sin ver mi `readiness_score` numérico crudo.
- Como alumno quiero saber qué skills dominé y cuáles me faltan.
- Como tutor quiero ver métricas detalladas de mis asignados (frecuencia, tasa de éxito, dominios estancados).
- Como sistema quiero registrar las sesiones del alumno (inicio, fin, duración).

Endpoints:

- `POST /sessions/start`
- `POST /sessions/{id}/end`
- `GET /students/{email}/sessions`
- `GET /students/{email}/progress` (vista motivacional para el alumno)
- `GET /students/{email}/metrics` (vista de seguimiento para el tutor)

Criterio de aceptación:

- El alumno ve "Vas por el 40%", "Dominaste 2 de 5 habilidades", "Te faltan 3 ejercicios para dominar SQL", "7 días seguidos aprendiendo".
- El alumno NO ve el `readiness_score` numérico.
- El tutor ve frecuencia de sesiones, duración promedio, días de inactividad, ratio teoría/práctica.

Esfuerzo estimado: 3-4 h.

Épica 2 — Tutores y asignaciones

Implementa el HITL del modelo: padrón de tutores + asignación a alumnos + dashboard.

Módulos a crear: `tutors` + `tutor_assignments`.

User stories:

- Como admin quiero registrar tutores con especialidades (skills que enseñan).
- Como admin quiero asignar un tutor a un alumno cuando se crea su plan.
- Como tutor quiero ver mi dashboard con todos mis alumnos asignados y su estado.
- Como tutor quiero ver el detalle de un alumno (progreso, attempts, recursos).

Endpoints:

- `POST /tutors`, `GET /tutors`, `GET /tutors/{id}`, `PATCH /tutors/{id}`
- `POST /tutor-assignments`
- `GET /tutors/{id}/dashboard` (resumen con todos los asignados)
- `GET /tutors/{id}/students/{student_email}` (detalle de un alumno)

Criterio de aceptación:

- Un tutor solo ve a sus alumnos asignados, nunca a alumnos de otros tutores.
- Un alumno tiene a lo sumo un tutor activo por path.
- El dashboard del tutor muestra alertas activas (cuando esté implementada la Épica 3).

Esfuerzo estimado: 4-5 h.

Épica 3 — Sistema de alertas

Detecta señales de riesgo de abandono o frustración y las muestra al tutor.

Módulo a crear: `alerts`.

Tipos de alerta:

Tipo	Trigger	Nivel
<code>LOW_PERSEVERANCE</code>	5 intentos fallidos seguidos en el mismo ejercicio	1
<code>ABANDON_RISK</code>	72hs sin actividad	2
<code>ABANDON_RISK</code>	7 días sin actividad	3 (escala a admin)
<code>LOST_PASSION</code>	Tiempo promedio de sesión cae >50% en 5 días	1
<code>ANOMALY_PATIENCE</code>	Lenguaje agresivo detectado por NLP (post-MVP)	2

User stories:

- Como sistema quiero generar alertas automáticamente cuando se cumplan los triggers.
- Como tutor quiero ver alertas activas de mis asignados, filtrables por nivel.
- Como tutor quiero marcar una alerta como resuelta tras intervenir.
- Como admin quiero recibir alertas nivel 3.

Endpoints:

- `GET /alerts?tutor_id=X&student_email=Y&resolved=false`
- `GET /alerts/{id}`
- `PATCH /alerts/{id}/resolve`

Implementación: generación síncrona dentro de `POST /attempts` (`LOW_PERSEVERANCE`) y un check de inactividad puede correr al consultar el dashboard del tutor (`ABANDON_RISK`).

Criterio de aceptación:

- El alumno NUNCA ve sus alertas.
- El tutor ve solo alertas de sus asignados.
- Una alerta resuelta queda registrada con `resolved_by_tutor_id`.

Esfuerzo estimado: 3-4 h.

Épica 4 — Next-unit logic y desbloqueo progresivo

Le da al alumno una ruta clara de qué hacer ahora y bloquea el avance hasta dominar.

User stories:

- Como alumno quiero saber qué unidad debo hacer ahora según mi progreso.
- Como sistema quiero bloquear las skills siguientes hasta dominar la actual (mastery $\geq$ 80%).
- Como alumno quiero recibir una explicación alternativa si no alcanzo el dominio tras N intentos en una skill.

Endpoints:

- `GET /learning-paths/{id}/next-unit` (devuelve la próxima unit pendiente según mastery)
- `PATCH /learning-paths/{id}/regenerate-unit` (genera contenido alternativo cuando hay bloqueo)

Criterio de aceptación:

- Si el alumno no terminó la unit de Pasión de Git, `next-unit` la devuelve.
- Si dominó Git al 80%, `next-unit` devuelve la primera unit de la siguiente skill.
- Si falló 5 veces seguidas, `regenerate-unit` produce contenido nuevo con prompt alternativo al LLM.

Esfuerzo estimado: 2-3 h.

Épica 5 — Validaciones críticas (entregables con ojo humano)

Algunos entregables requieren juicio humano (presentaciones, code review de proyectos).

Módulo a crear: `validations`.

User stories:

- Como alumno quiero subir un entregable (link a repo, video, presentación) asociado a una unit.
- Como tutor quiero ver entregables pendientes de mis asignados.
- Como tutor quiero aprobar/rechazar un entregable con feedback escrito.
- Como alumno quiero ver el feedback de mis entregables.

Endpoints:

- `POST /validations` (alumno sube link)
- `GET /tutors/{id}/pending-validations`
- `PATCH /validations/{id}/validate` (tutor aprueba/rechaza con feedback)

Esfuerzo estimado: 3 h.

Épica 6 — Auth (fase mock → JWT real)

Activa la matriz de accesos (`access-matrix.md`) que hoy está documentada pero no aplicada. La épica se entrega en dos fases: primero headers mock para destrabar el resto del sistema, después JWT real

cuando se necesite producción.

Fase 6A — Headers mock (MVP)

Permite desarrollar y demostrar permisos por rol sin todavía implementar la criptografía. Cuando llegue Fase 6B, el resto del código no se entera.

User stories:

- Como sistema quiero leer `X-User-Id` y `X-User-Email` y `X-User-Role` de los headers HTTP para identificar al usuario actual.
- Como sistema quiero rechazar endpoints según rol declarado en cada router.
- Como dev quiero un único punto donde se decide quién es el usuario actual, para poder reemplazarlo después por JWT sin tocar los routers.

Cambios:

- Crear módulo `app/modules/auth/` :
- `dependencies.py` — define `get_current_user` , `require_role(roles)` , `require_self_or_admin(...)` .
- `schemas.py` — define `AuthenticatedUser` (id, email, role).
- En `dependencies.get_current_user` : lee headers, busca al user en `users` (verifica que el id existe y el rol coincide), devuelve `AuthenticatedUser` . Si falta header o no existe el user → `401` . Si el rol no matchea con el del header → `401` .
- Decorar endpoints con `Depends(require_role(...))` :
- `POST /tutors` → solo `admin` .
- `GET /tutors/{id}/dashboard` → `tutor` (solo si `id` coincide con el suyo) o `admin` .
- `POST /attempts` → `student` (solo sobre planes propios).
- `GET /alerts` → `tutor` (solo de sus asignados) o `admin` .

Criterio de aceptación:

- `POST /tutors` sin headers → `401 Unauthorized` .
- `POST /tutors` con `X-User-Role=student` → `403 Forbidden` .
- `GET /tutors/{id}/dashboard` con `X-User-Id` distinto al `id` del path → `403` .
- `POST /attempts` con `student_email` distinto al `X-User-Email` → `403` (ya está implementado en el service, falta blindar con el header).
- Toda la matriz de `access-matrix.md` queda aplicable con un solo cambio de dependency.

Esfuerzo estimado: 3-4 h.

Fase 6B — JWT real (post-MVP, antes de presentar)

Reemplaza los headers mock por tokens firmados. El resto del código no cambia: solo el contenido de `get_current_user` .

User stories:

- Como alumno/tutor/admin quiero registrarme con email + password y recibir un access token.
- Como usuario autenticado quiero refrescar mi token sin volver a loguearme.
- Como sistema quiero almacenar las passwords hasheadas, nunca en plaintext.
- Como sistema quiero invalidar tokens (logout) ante un evento de seguridad.

Endpoints nuevos:

- `POST /auth/register` — recibe `email`, `password`, `first_name`, `last_name`, `role`. Crea el user con `password_hash`. Devuelve `access_token` + `refresh_token`.
- `POST /auth/login` — recibe `email` + `password`. Devuelve los dos tokens.
- `POST /auth/refresh` — recibe `refresh_token`. Devuelve nuevo `access_token`.
- `POST /auth/logout` — invalida el refresh token del usuario actual.
- `POST /auth/change-password` — recibe password actual + nueva.

Modelo de datos:

- Agregar columna `password_hash` a la tabla `users`.
- Tabla nueva `refresh_tokens` (id, user_id, token_hash, expires_at, revoked_at).

Decisiones técnicas:

- **Hashing:** `passlib[bcrypt]` o `argon2-cffi`. Default bcrypt.
- **Tokens:** `pyjwt` con HS256. Secret en env var `JWT_SECRET_KEY` (mínimo 32 chars random, rotatable).
- **Lifetimes:** access token 15 min, refresh token 7 días.
- **Storage del refresh token:** se devuelve al cliente, se guarda hashado en BD para poder revocarlo.
- **Cliente:** envía `Authorization: Bearer <access_token>` en cada request.
- **Compatibilidad con Fase 6A:** durante la migración, `get_current_user` acepta tanto JWT como headers mock (controlado por env var `AUTH_MODE=mock|jwt`).

Cambios en `get_current_user`:

```
def get_current_user(authorization: str = Header(None)) -> AuthenticatedUser:
    if settings.auth_mode == "mock":
        # Fase 6A: lee X-User-Id, X-User-Email, X-User-Role
        ...
    else:
        # Fase 6B: parsea el JWT y valida firma + expiración
        token = authorization.removeprefix("Bearer ").strip()
        payload = jwt.decode(token, settings.jwt_secret_key, algorithms=["HS256"])
        return AuthenticatedUser(...)
```

Criterio de aceptación:

- `POST /auth/register` crea user con `password_hash` (nunca plaintext en BD).
- `POST /auth/login` con credenciales válidas devuelve dos tokens.
- Request sin `Authorization` header → `401`.
- Request con token expirado → `401`.
- Request con token modificado (firma inválida) → `401`.
- `POST /auth/refresh` con refresh token revocado → `401`.
- Cambiar `AUTH_MODE=mock` en Render permite volver al modo legacy sin redeploy.

Variables de entorno nuevas:

Variable	Valores	Propósito
<code>AUTH_MODE</code>	<code>mock</code> (default) / <code>jwt</code>	Cuál mecanismo de auth está activo

Variable	Valores	Propósito
<code>JWT_SECRET_KEY</code>	string aleatorio	Secret para firmar tokens. Rotar al deployar producción real
<code>JWT_ACCESS_TTL_MINUTES</code>	int (default 15)	Lifetime del access token
<code>JWT_REFRESH_TTL_DAYS</code>	int (default 7)	Lifetime del refresh token

Riesgos / decisiones pendientes:

- ¿Permitimos auto-registro de cualquier rol o solo `student`? Probable: solo `student`; tutores y `company_admin` los crea el admin.
- ¿Email verification? Post-MVP. Por ahora se asume que el email es válido.
- ¿Rate limiting en login? Post-MVP, vía middleware (slowapi).
- ¿OAuth con Google? Útil porque los alumnos ya tienen cuenta Google (Equipo 1 usa form de Google para onboarding). Sería una **Épica 9 separada** si avanza.

Esfuerzo estimado: 6-8 h.

Por qué dividir en fases

La Fase 6A desbloquea el resto del backlog (Tutores, Alertas, Validaciones todas necesitan saber quién es el usuario actual). Hacer JWT directo desde el día 1 retrasa la entrega del TP.

La separación también prueba que el patrón de `Depends(get_current_user)` es el correcto: si la Fase 6B se hace bien, **ningún router cambia** entre las dos fases.

Épica 7 — Eventos hacia Equipo 1

Implementa el contrato Equipo 2 → Equipo 1 definido en `scope-equipos.md`.

User stories:

- Como sistema quiero emitir un evento append-only cada vez que algo relevante pasa en el ciclo del alumno.
- Como Equipo 1 quiero leer la tabla `events` para calcular métricas globales.

Eventos a emitir:

```
session_started, session_ended
exercise_attempted, exercise_completed
unit_completed, module_completed, path_completed
mastery_achieved (skill llega al 80%)
alert_raised, tutor_intervened
```

Cambios:

- Tabla `events` con `student_email`, `type`, `payload_json`, `created_at`.
- Helper `events.emit(type, payload)` llamado desde los services existentes (`attempts.create_attempt`, `sessions.end`, etc.).

Esfuerzo estimado: 2 h.

Épica 8 — Calidad técnica

User stories transversales que no agregan funcionalidad pero suben calidad del entregable.

Items:

- Tests unitarios y de integración con `pytest` y `httpx`. Cobertura mínima del flujo de happy path en cada módulo.
- CORS habilitado para frontend en otro dominio (`fastapi.middleware.cors.CORSMiddleware`).
- Migración de `Base.metadata.create_all` a Alembic para gestionar cambios de schema sin perder datos.

Esfuerzo estimado: 4-6 h total (cada item ~1-2 h).